

Università degli Studi di Padova
DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”
CORSO DI LAUREA IN INFORMATICA



**Evoluzione e ottimizzazione di un firmware embedded per
sistema di telemetria basato su ESP32**

Tesi di Laurea

Relatore
Prof. Zanella Marco

Laureando
Laoud Zakaria
Matricola 2113196

إِنَّ مَعَ الْعُسْرِ يُسْرًا

“In verità, con la difficoltà giunge la facilità.”

— Corano(96 - 6)

Ringraziamenti

Desidero esprimere la mia sincera gratitudine al professor Zanella Marco, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato (da scrivere meglio)

Ricorda di ringraziare il prof cavallin.

Padova, Settembre 2026

Laoud Zakaria

Sommario

Il presente lavoro di tesi riguarda l'evoluzione e l'ottimizzazione di un firmware embedded sviluppato su piattaforma ESP32 per un sistema di telemetria applicato al settore motorsport. L'obiettivo principale è il miglioramento dell'efficienza, dell'affidabilità e delle funzionalità del sistema attraverso interventi sulla gestione dei dati, sull'acquisizione delle informazioni e sull'ottimizzazione delle risorse hardware disponibili.

Il lavoro si concentra sull'analisi del firmware esistente e sull'introduzione di miglioramenti volti a rendere il sistema più performante e adattabile alle esigenze di acquisizione dati in ambito motociclistico. Verranno inoltre integrate nuove funzionalità per ampliare le capacità del dispositivo e migliorare la qualità delle informazioni raccolte durante l'utilizzo.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	1
1.3	Motivazioni della scelta del progetto	2
1.4	Convenzioni tipografiche	3
2	Descrizione dello stage	4
2.1	Competenze e obiettivi	4
2.1.1	Competenze da acquisire	4
2.1.2	Obiettivi	5
2.1.2.1	Obiettivi obbligatori	5
2.1.2.2	Obiettivi desiderabili	6
2.1.2.3	Obiettivi facoltativi	6
2.2	Pianificazione	6
2.3	Organizzazione del lavoro	8
2.4	Analisi preventiva dei rischi	8
3	Analisi dei requisiti	10
3.1	Stato attuale del firmware	10
3.2	Analisi degli utenti	10
3.3	Lista degli <i>use case</i>	10
3.3.1	UC1-F – Acquisizione binaria dei dati sensore	11
3.3.2	UC2-F – Registrazione dei dati binari dei sensori su SD	12
3.3.3	UC3-F – Calcolo dello spazio occupato dai dati registrati	13
3.3.4	UC4-A – Visualizzazione dello spazio di archiviazione occupato	14
3.3.5	UC5-F – Gestione della capacità di registrazione e blocco automatico	15
3.3.6	UC6-A – Notifica di sessione bloccata per spazio GPS esaurito	15
3.3.7	UC7-F – Recupero della sessione dopo rimozione SD	16
3.3.8	UC8-F – Notifica di rimozione della scheda microSD	16
3.3.9	UC9-A – Gestione della rimozione SD durante la sessione	17
3.3.10	UC10-A – Gestione dell'assenza della SD all'avvio sessione	18
3.3.11	UC11-F – Acquisizione dati dal bus CAN	18
3.3.12	UC12-F – Gestione dello stato del firmware tramite LED_G	20
3.3.12.1	UC12.1-F – Segnalazione di scheda microSD mancante	21
3.3.12.2	UC12.2-F – Segnalazione di errore generico	22
3.3.12.3	UC12.3-F – Segnalazione di pairing WiFi	22
3.3.12.4	UC12.4-F – Segnalazione di pairing BLE	23
3.3.12.5	UC12.5-F – Segnalazione di recupero dati per calibrazione	23
3.3.12.6	UC12.6-F – Segnalazione di calibrazione in corso	24

3.3.12.7	UC12.7-F – Segnalazione di download dati	24
3.3.12.8	UC12.8-F – Segnalazione di eliminazione dati	25
3.3.12.9	UC12.9-F – Segnalazione di sospensione per inattività	25
3.3.12.10	UC12.10-F – Segnalazione di logging attivo	26
3.3.12.11	UC12.11-F – Segnalazione di dispositivo avviato in attesa di sessione	26
3.3.13	UC13-F – Sospensione dell’acquisizione sensori in stato di inattività	27
3.3.14	UC14-A – Configurazione del tempo di inattività	28
3.3.15	UC15-F – Avvio e interruzione sessione tramite comandi BLE	28
3.3.16	UC16-F – Calibrazione dell’IMU	29
3.3.17	UC17-A – Sequenza guidata di calibrazione dell’IMU	30
3.3.18	UC18-A – Visualizzazione dei dati calibrati	30
3.3.19	UC19-A – Visualizzazione dei movimenti della moto in tempo reale	31
3.3.20	UC20-F – Calibrazione automatica dell’altitudine a inizio sessione	32
3.3.21	UC21-F – Salvataggio delle calibrazioni collegate alla sessione	32
3.4	Tracciamento dei requisiti	33
3.4.1	Requisiti funzionali	33
3.4.2	Requisiti non funzionali	35
3.4.3	Requisiti di vincolo	36
3.4.4	Tracciamento Fonti – Requisiti	37
3.4.5	Riepilogo dei requisiti	38
4	Tecnologie utilizzate	39
4.1	ESP32-S3-DevKitC-1	39
4.2	ESP-WROOM-32	40
4.3	C++20	41
4.4	ESP32 Arduino Core	41
4.5	FreeRTOS	42
4.5.1	Sistemi real-time e concorrenza	42
4.5.2	Scheduler e priorità	42
4.5.3	Task	43
4.5.4	Stati delle task	43
4.5.5	Race condition e sincronizzazione	44
4.5.6	Mutex	45
4.5.7	Comunicazione tra task	45
4.6	Expo & React Native	46
4.7	TypeScript	46
4.8	Git	47
4.9	PlatformIO	47
4.10	L ^A T _E X	47
4.11	PlantUML	48
5	Progettazione e codifica	49
5.1	Tecnologie e strumenti	49
5.2	Ciclo di vita del software	49

5.3	Progettazione	49
5.3.1	Namespace 1	49
5.4	Design Pattern utilizzati	49
5.5	Codifica	49
6	Verifica e validazione	50
7	Conclusioni	51
7.1	Consuntivo finale	51
7.2	Raggiungimento degli obiettivi	51
7.3	Conoscenze acquisite	51
7.4	Valutazione personale	51
	Glossario	i
	Bibliografia	v
	Sitografia	vi

Elenco delle figure

1.1	Logo dell'azienda Eggon Srl	1
3.1	UC1-F – Acquisizione binaria dei dati sensore	11
3.2	Inclusioni UC2-F – Registrazione dei dati sensore su SD	12
3.3	UC2-F – Registrazione dei dati sensore su SD	13
3.4	UC3-F – Calcolo dello spazio occupato dai dati registrati	13
3.5	UC4-A – Visualizzazione dello spazio di archiviazione occupato	14
3.6	UC5-F – Gestione della capacità di registrazione e blocco automatico	15
3.7	UC6-A – Notifica di sessione bloccata per spazio GPS esaurito	15
3.8	UC7-F – Recupero della sessione dopo rimozione SD	16
3.9	UC8-F – Notifica di rimozione della scheda microSD	16
3.10	UC9-A – Gestione della rimozione SD durante la sessione	17
3.11	UC10-A – Gestione dell'assenza della SD all'avvio sessione	18
3.12	UC11-F – Acquisizione dati dal CAN	18
3.13	UC12-F – Gestione dello stato del firmware tramite LED	20
3.14	UC12.1-F – Segnalazione di scheda microSD mancante	21
3.15	UC12.2-F – Segnalazione di errore generico	22
3.16	UC12.3-F – Segnalazione di pairing WiFi	22
3.17	UC12.4-F – Segnalazione di pairing BLE	23
3.18	UC12.5-F – Segnalazione di recupero dati per calibrazione	23
3.19	UC12.6-F – Segnalazione di calibrazione in corso	24
3.20	UC12.7-F – Segnalazione di download dati	24
3.21	UC12.8-F – Segnalazione di eliminazione dati	25
3.22	UC12.9-F – Segnalazione di sospensione per inattività	25
3.23	UC12.10-F – Segnalazione di logging attivo	26
3.24	UC12.11-F – Segnalazione di dispositivo avviato in attesa di sessione	26
3.25	UC13-F – Sospensione dell'acquisizione sensori in stato di inattività	27
3.26	UC14-A – Configurazione del tempo di inattività	28
3.27	UC15-F – Avvio e interruzione sessione tramite comandi BLE	28
3.28	UC16-F – Calibrazione dell'IMU	29
3.29	UC17-A – Sequenza guidata di calibrazione dell'IMU	30
3.30	UC18-A – Visualizzazione dei dati calibrati	30
3.31	UC19-A – Visualizzazione dei movimenti della moto in tempo reale	31
3.32	UC20-F – Calibrazione automatica dell'altitudine a inizio sessione	32
3.33	UC21-F – Salvataggio delle calibrazioni collegate alla sessione	32
4.1	Scheda di sviluppo ESP32-S3-DevKitC-1	39
4.2	Modulo ESP-WROOM-32	40
4.3	Logo del linguaggio C++	41

4.4	FreeRTOS	42
4.5	Logo di Expo & React Native	46
4.6	Git	47
4.7	PlatformIO	47
4.8	Esempio di diagramma generato con PlantUML	48

Elenco delle tabelle

3.1	Tracciamento dei requisiti funzionali.	35
3.2	Tracciamento dei requisiti non funzionali.	36
3.3	Tracciamento dei requisiti di vincolo.	37
3.4	Tracciamento Fonti – Requisiti.	38
3.5	Conteggio dei requisiti tracciati.	38
4.1	Principali parametri di configurazione di una task in FreeRTOS	43
4.2	Principali stati di una task in FreeRTOS	44
4.3	Esempio di interleaving che genera una race condition	44
4.4	Fasi di utilizzo di un mutex	45
4.5	Confronto tra queue e task notification in FreeRTOS	45

Elenco dei codici sorgenti

1.1	Fibonacci recursive	3
-----	-------------------------------	---

1. Introduzione

Nel seguente capitolo viene presettata l'azienda ospitante, il progetto svolto e le motivazioni che hanno portato alla scelta di questo argomento per il lavoro di tesi.

1.1 L'azienda

Eggon S.r.l. è una *software house* con sede a Padova che opera in diversi ambiti tecnologici, sviluppando soluzioni software innovative e fornendo servizi personalizzati in base alle esigenze dei propri clienti. L'attività dell'azienda si concentra principalmente in settori quali la *Digital Health_G*, l'*Industria 5.0_G* con dispositivi correlati all'*Industrial Internet of Things_G*, l'*Insurtech_G* e il *Fintech_G*, oltre che nell'ambito dell'automazione logistica.



Figura 1.1: Logo dell'azienda Eggon Srl

La società nasce dall'esperienza maturata negli Stati Uniti dai suoi fondatori, Gianpaolo Ferrarin e Fulvio Menegozzo, i quali hanno successivamente introdotto nel contesto italiano metodologie di lavoro, approcci innovativi e una forte attenzione alla concretezza nello sviluppo di soluzioni tecnologiche.

1.2 Il progetto

Il tirocinio si è svolto presso Eggon Srl ed è stato dedicato allo sviluppo di **EggLog Motion Core**, un ecosistema per la telemetria applicata al motorsport motociclistico. Il sistema si articola in un dispositivo *Embedded_G* per l'acquisizione dei dati a bordo del veicolo e in un'applicazione dedicata alla consultazione, alla visualizzazione e all'analisi delle informazioni raccolte durante le sessioni di guida stradale o in pista.

L'obiettivo del progetto è stato realizzare un *Firmware_G* affidabile per il dispositivo embedded, basato su microcontrollore ESP32 (vd. paragrafo ESP32), in grado di acquisire i dati provenienti dai diversi sensori installati a bordo, organizzarli in modo efficiente e memorizzarli localmente su scheda microSD, garantendo continuità di acquisizione, integrità dei dati e prestazioni adeguate ai vincoli tipici di un sistema real-time. Parallelamente, è stata sviluppata un'applicazione destinata all'interazione con il dispositivo e all'analisi

dei dati acquisiti.

Il lavoro ha previsto inoltre lo studio del prototipo realizzato dal precedente gruppo di tirocinanti, con l'obiettivo di valutare le soluzioni adottate e individuare possibili margini di miglioramento, sia a livello di architettura software sia in vista di una futura revisione hardware del dispositivo.

Il progetto ha permesso di affrontare problematiche caratteristiche dello sviluppo di sistemi embedded destinati ad applicazioni reali, approfondendo aspetti relativi all'acquisizione dei dati da sensori, alla gestione efficiente delle risorse hardware e alla progettazione di firmware affidabili.

1.3 Motivazioni della scelta del progetto

Le principali motivazioni che mi hanno portato a scegliere questo progetto di tirocinio sono:

- **Interesse per l'Internet of Things e i sistemi embedded**

Fin dagli anni della scuola secondaria di secondo grado ho sviluppato un forte interesse per il mondo dell'*Internet of Things_G*, grazie alla realizzazione di alcuni progetti che mi hanno permesso di avvicinarmi ai sistemi embedded. Questo interesse mi ha spinto a ricercare un'esperienza che mi consentisse di approfondire tali tematiche in un contesto professionale.

- **Interesse per l'automotive**

Nel corso degli anni ho sviluppato un particolare interesse per il settore *Automotive_G* e per le tecnologie che ne caratterizzano l'evoluzione, come i sistemi di telemetria, le reti di comunicazione veicolari e le applicazioni software dedicate all'acquisizione e all'analisi dei dati.

- **Ambiente di lavoro giovane**

Un ulteriore elemento che ha influenzato la mia scelta è stato il contesto aziendale. Essendo composta prevalentemente da un team giovane, l'azienda favorisce una comunicazione diretta, il confronto continuo e uno scambio di idee privo di barriere.

- **Approfondimento dello sviluppo firmware**

Nel corso di precedenti esperienze avevo già utilizzato piattaforme come *Arduino_G*, ma esclusivamente per applicazioni relativamente semplici. Il tirocinio rappresentava quindi un'opportunità per affrontare uno sviluppo firmware più strutturato, lavorando direttamente su un prodotto reale e acquisendo competenze più avanzate nel campo dell'embedded.

- **Crescita personale**

Gli argomenti relativi alle telecomunicazioni sono sempre stati tra quelli che ho trovato più complessi durante il percorso scolastico. Per questo motivo ho visto il progetto come una sfida personale, con l'obiettivo di consolidare le mie conoscenze e migliorare le mie competenze.

1.4 Convenzioni tipografiche

Durante la redazione del presente documento sono state adottate alcune convenzioni tipografiche con l'obiettivo di rendere più chiara e uniforme la consultazione del testo. In particolare:

- Gli acronimi, le abbreviazioni e i termini tecnici o di uso non comune vengono descritti all'interno del *glossario*, riportato nella parte finale del documento;
- Alla prima occorrenza di un termine presente nel glossario viene utilizzata la seguente nomenclatura: *Arduino_G*;
- Nel caso degli acronimi alla prima ricorrenza viene utilizzato il riferimento per esteso (*Industrial Internet of Things_G*), mentre dalla seconda in poi viene semplicemente riportato come acronimo (IIoT);
- I termini in lingua straniera non entrati nell'uso comune della lingua italiana, così come quelli appartenenti al gergo tecnico, vengono riportati in carattere *corsivo*;
- I nomi di funzioni, variabili o altri elementi appartenenti al codice sorgente vengono rappresentati mediante carattere **monospaziato**;
- I riferimenti a libri, articoli o altre risorse presenti nella bibliografia (*bibliografia*) sono indicati tramite il relativo identificativo numerico, ad esempio *Manifesto Agile*. URL: <http://agilemanifesto.org/iso/it/>;
- Gli estratti di codice sorgente vengono riportati attraverso appositi blocchi formatati, come riportato di seguito:

```
def recur_fibo(n):  
    if n <= 1:  
        return n  
    else:  
        return(recur_fibo(n-1) + recur_fibo(n-2))  
nterms = 10  
if nterms <= 0:  
    print("Plese enter a positive integer")  
else:  
    print("Fibonacci sequence:")  
    for i in range(nterms):  
        print(recur_fibo(i))
```

Codice 1.1: Fibonacci recursive

2. Descrizione dello stage

Il presente capitolo descrive il percorso di stage svolto presso l'azienda, illustrando gli obiettivi formativi, le competenze richieste e le attività pianificate durante il periodo di tirocinio. Vengono inoltre presentate l'organizzazione del lavoro adottata, l'analisi preventiva dei principali rischi progettuali e il ruolo svolto dal tutor aziendale nel supporto alle attività di sviluppo.

2.1 Competenze e obiettivi

L'obiettivo principale dello stage è stato l'evoluzione e l'ottimizzazione del firmware embedded del sistema *EggLog Motion Core*, una piattaforma dedicata all'acquisizione e alla gestione di dati telemetrici in ambito motorsport, sviluppata su piattaforma ESP32.

Il lavoro riguarda principalmente l'analisi dell'architettura firmware esistente, il miglioramento dei componenti software già presenti, sia su firmware che nell'applicazione mobile, e l'introduzione di nuove funzionalità volte a rendere il sistema più affidabile, efficiente e facilmente estendibile.

Le attività svolte hanno coinvolto diversi aspetti del sistema, tra cui la gestione dello *storage* su memoria microSD, l'ottimizzazione dei servizi di comunicazione BLE (*Bluetooth Low Energy_G*), la gestione del trasferimento dei dati acquisiti real-time e l'analisi del traffico CAN proveniente dalla centralina elettronica (*Electronic Control Unit_G* detta ECU) della moto mediante tecniche di *CAN sniffing_G*.

2.1.1 Competenze da acquisire

Durante il periodo di tirocinio erano previste l'acquisizione e il consolidamento delle seguenti competenze:

- sviluppo e manutenzione di firmware embedded in linguaggio *C++_G* su piattaforma ESP32;
- analisi, *refactoring* e ottimizzazione di codice embedded esistente;
- gestione di sistemi di memorizzazione embedded e ottimizzazione delle operazioni di lettura e scrittura su memoria SD;
- sviluppo e ottimizzazione di servizi BLE;
- analisi dei protocolli di comunicazione utilizzati in ambito automotive, con particolare riferimento al *bus CAN_G*;
- acquisizione e analisi di messaggi dal bus CAN provenienti dalla ECU di un veicolo motorizzato;

- integrazione lato applicazione mobile delle nuove funzionalità sviluppate lato firmware, attraverso l'implementazione delle relative funzionalità di comunicazione e gestione dei dati;
- utilizzo di strumenti di *debugging* e *testing* per sistemi embedded;
- produzione di documentazione tecnica relativa al firmware e ai protocolli di comunicazione analizzati.

2.1.2 Obiettivi

Gli obiettivi definiti dall'azienda sono stati classificati secondo tre livelli di priorità:

- **O**: obbligatorio;
- **D**: desiderabile;
- **F**: facoltativo.

2.1.2.1 Obiettivi obbligatori

Gli obiettivi obbligatori comprendevano principalmente attività volte al miglioramento dell'affidabilità e delle prestazioni del firmware esistente.

- **OO-1**: *Refactoring* e ottimizzazione del sistema di lettura e scrittura su scheda SD;
- **OO-2**: Completamento del sistema di rotazione dei file mediante l'utilizzo di ring-buffer;
- **OO-3**: Implementare la scrittura e salvataggio dei dati raccolti direttamente in formato binario;
- **OO-4**: Implementazione di un sistema di *recovery* che permette di riprendere la scrittura su SD in caso di guasti o rimozione della stessa del suo *slot*.
- **OO-5**: Miglioramento dei servizi BLE e introduzione di nuovi comandi firmware;
- **OO-6**: Aggiunta di metriche di confronto per poter stimare il miglioramento rispetto alla versione precedente del firmware;
- **OO-7**: Implementare un protocollo di calibrazione dei sensori;
- **OO-8**: Aggiungere una logica di sessionamento, attivando i sensori solamente all'avvio della sessione e disattivandoli quando termina la sessione;
- **OO-9**: Produzione della documentazione tecnica relativa al firmware sviluppato.

2.1.2.2 Obiettivi desiderabili

Gli obiettivi desiderabili erano principalmente orientati all'introduzione di funzionalità aggiuntive e al miglioramento delle modalità di trasferimento dei dati.

- **OD-1:** Migliorare la visualizzazione dei dati in tempo reale sul dispositivo mobile;
- **OD-2:** Permettere la visualizzazione della moto in tempo reale con un modellino 3D;
- **OD-3:** Integrazione dell'analisi e gestione dei dati provenienti dalla ECU;

2.1.2.3 Obiettivi facoltativi

- **OF-1:** Implementazione di un sistema avanzato di indicizzazione delle sessioni memorizzate sulla scheda SD.
- **OF-2:** Implementazione di un sistema di **intactivity** per permettere la disattivazione dei sensori quando non viene rilevato alcun movimento per un tempo configurabile dall'utente.

2.2 Pianificazione

Le attività di stage sono state pianificate con il tutor aziendale considerando un monte complessivo di 320 ore, distribuite su otto settimane lavorative da quaranta ore ciascuna. Tali settimane sono state suddivise come segue:

1. Prima settimana – *Onboarding* e analisi del progetto

- Introduzione al sistema **EggLog Motion Core** e all'architettura generale della piattaforma;
- Configurazione dell'ambiente di sviluppo e degli strumenti di compilazione e debugging;
- Analisi della struttura del firmware, dell'app esistente e delle principali componenti software e hardware;
- Studio delle modalità di acquisizione, memorizzazione e trasferimento dei dati.

2. Seconda settimana – Studio delle tecnologie e analisi del firmware

- Approfondimento della gestione della memoria microSD e del relativo *file system*;
- Analisi dello stack BLE e dei servizi implementati;
- Studio dei protocolli di comunicazione utilizzati dal sistema;
- Individuazione delle principali criticità presenti nel firmware.

3. Terza settimana – Analisi e progettazione delle modifiche

- Analisi dell'architettura software esistente e definizione degli interventi di *refactoring*;

- Progettazione delle modifiche relative alla gestione dello storage;
- Studio dei meccanismi di recupero e gestione dei dati in caso di errore;
- Analisi preliminare del bus CAN della moto e delle informazioni trasmesse dalla ECU.

4. Quarta settimana – Sviluppo e ottimizzazione dello storage

- *Refactoring* dei moduli responsabili della gestione della memoria SD;
- Ottimizzazione delle operazioni di lettura e scrittura dei dati;
- Miglioramento del sistema di rotazione dei file;
- Implementazione e verifica dei meccanismi di *recovery*.

5. Quinta settimana – Gestione dei dati e comunicazioni

- Implementazione delle funzioni di calibrazione dei sensori;
- Miglioramento della qualità dei dati inviati via BLE, inteso come invio di dati già calibrati;
- Miglioramento dei servizi BLE;
- Implementazione di un sistema di inattività, basato su GPS con *fallback* su accelerometro.

6. Sesta settimana – Analisi dei pacchetti CAN e integrazione firmware

- Acquisizione dei messaggi presenti sul bus CAN tramite attività di *sniffing*;
- Analisi degli identificativi e degli identificativi dei pacchetti CAN provenienti dalla ECU;
- Studio della possibile integrazione delle informazioni trasmesse nel bus CAN all'interno del sistema telemetrico;

7. Settima settimana – Testing e validazione

- Esecuzione di test funzionali sul firmware modificato;
- Raccolta di metriche sulle prestazioni del sistema;
- Esecuzione di benchmark sulle operazioni di memorizzazione e occupazione della CPU;
- Analisi e risoluzione delle problematiche emerse durante i test.

8. Ottava settimana – Documentazione finale

- Aggiornamento della documentazione tecnica del firmware;
- Descrizione delle modifiche architetturali introdotte;
- Documentazione relativa ai protocolli di comunicazione analizzati;
- Stesura della relazione finale di stage.

2.3 Organizzazione del lavoro

Le attività di sviluppo sono state organizzate seguendo una metodologia *Metodologia Agile_G*, ispirata al framework *Scrum_G*. Il lavoro è stato suddiviso in sprint della durata indicativa di due settimane, durante i quali venivano pianificate le attività da svolgere, definiti gli obiettivi da raggiungere e monitorato lo stato di avanzamento delle attività insieme al tutor aziendale.

Durante il periodo di stage sono stati inoltre svolti degli *Stand-up meeting_G* con cadenza giornaliera, finalizzati al mantenimento dell'allineamento tra i membri del team. Questi incontri hanno permesso di condividere i progressi effettuati, discutere eventuali problematiche emerse durante le fasi di sviluppo e progettazione e individuare possibili soluzioni attraverso il confronto tecnico.

Il gruppo di lavoro era composto da due tirocinanti, ciascuno responsabile dello sviluppo di una specifica componente del sistema. Il sottoscritto ha operato principalmente sulla componente firmware embedded, occupandosi dell'evoluzione del software a bordo del dispositivo, mentre la collega ha lavorato sulle componenti applicative del sistema.

Nonostante la suddivisione delle responsabilità, le attività sono state svolte in maniera collaborativa, favorendo il confronto continuo tra i membri del team e garantendo una corretta integrazione tra firmware, applicazione mobile e infrastruttura software.

2.4 Analisi preventiva dei rischi

Prima dell'avvio delle attività è stata effettuata un'analisi preventiva dei principali rischi che avrebbero potuto influenzare il corretto svolgimento del progetto. Per ciascun rischio individuato si riportano:

- la probabilità di accadimento (Improbabile, Poco probabile, Probabile, Altamente probabile);
- le conseguenze in caso di accadimento (Lieve, Medio, Grave, Gravissimo);
- rimedi adottati per mitigarne la probabilità o le conseguenze.

I possibili rischi individuati sono:

- **Introduzione di regressioni nel codice esistente**

Probabilità: Probabile – il firmware esistente non dispone di test automatici, per cui le modifiche apportate al codice possono alterare comportamenti già funzionanti senza che ciò venga rilevato immediatamente.

Conseguenze: Grave – una regressione non rilevata può compromettere l'affidabilità dell'acquisizione o della scrittura dei dati, richiedendo una nuova sessione di test per essere individuata.

Rimedi: prima di intervenire su ciascun modulo è stata condotta un'analisi puntuale dell'architettura e del codice preesistente; dopo ogni modifica sono state eseguite sessioni di test manuale mirate alle funzionalità potenzialmente impattate, seguite da revisione del codice con il tutor aziendale.

- **Mancato miglioramento delle prestazioni**

Probabilità: Probabile – le modifiche introdotte per ottimizzare acquisizione, buffering e trasferimento dei dati comportano un *overhead* computazionale e di occupazione *RAM_G* che potrebbe annullare, in tutto o in parte, il guadagno prestazionale atteso, specialmente se il compromesso tra velocità e memoria non viene bilanciato correttamente.

Conseguenze: Medio – un mancato miglioramento prestazionale vanificherebbe l'obiettivo stesso delle ottimizzazioni introdotte durante il tirocinio, richiedendo ulteriori interventi correttivi.

Rimedi: per prevenire il rischio, la scelta delle strutture dati e degli algoritmi di buffering è stata guidata da una stima preventiva del loro costo computazionale e di memoria, con verifiche incrementalmente ad ogni modulo sviluppato anziché un'unica misurazione finale. Per individuare tempestivamente eventuali regressioni prestazionali sono stati inoltre implementati dei test di *benchmark* che misurano: il tempo impiegato per ogni operazione di scrittura, la quantità di dati scritti e il livello di *RAM_G* occupato dal sistema di buffering (cfr. ?? RICORDATI DI AGGIUNGERE IL RIFERIMENTO). Tali *benchmark* sono stati implementati anche nella versione precedente del firmware a scopo di confronto.

- **Interpretazione errata dei messaggi provenienti dal bus CAN**

Probabilità: Altamente probabile – i messaggi trasmessi sul bus CAN dalla ECU della moto non seguono una codifica standardizzata e pubblica, per cui è necessario ricostruirne il significato tramite osservazione ed ipotesi, con concreto rischio di misinterpretazione.

Conseguenze: Grave – un'errata interpretazione porta all'acquisizione di dati non corretti o non significativi, invalidando le misurazioni raccolte e richiedendo di ripetere l'analisi.

Rimedi: prima di avviare l'attività di *sniffing* è stata condotta un'analisi di realizzabilità software; l'interpretazione dei messaggi è stata poi validata in modo incrementale confrontando i valori decodificati con misurazioni di riferimento note.

- **Incompatibilità tra le componenti sviluppate in parallelo**

Probabilità: Probabile – il firmware e l'applicazione mobile sono stati sviluppati contemporaneamente da persone diverse, con il rischio che le rispettive interfacce evolvano in modo incoerente tra loro.

Conseguenze: Medio – un eventuale disallineamento richiederebbe comunque lavoro aggiuntivo di adattamento e potrebbe ritardare il completamento degli obiettivi predisposti.

Rimedi: per le funzionalità introdotte nel corso del tirocinio, le modifiche lato firmware e la relativa integrazione lato applicativo sono state implementate in modo contestuale dalla stessa persona, garantendo coerenza tra le interfacce senza necessità di sincronizzazione con altri sviluppatori; sono stati inoltre organizzati confronti periodici con il tutor aziendale per validare le scelte effettuate.

3. Analisi dei requisiti

Questo capitolo presenta gli use case rilevanti per le attività di ottimizzazione ed evoluzione svolte sul firmware embedded del dispositivo EggLog Motion Core e dell'applicazione mobile (d'ora in poi solo EggLog e EggLog App) durante il periodo di tirocinio, insieme alla relativa analisi dei requisiti.

3.1 Stato attuale del firmware

Allo stato attuale, il firmware di EggLog è in grado di acquisire i dati provenienti dai sensori di bordo (*IMU_G*, magnetometro, barometro e modulo GPS). I dati acquisiti vengono temporaneamente memorizzati in buffer in RAM secondo uno schema di *Ping-pong buffering_G* e successivamente scritti in formato testuale (*CSV_G*) su scheda microSD, organizzati in file distinti all'interno di cartelle dedicate a ciascun sensore.

Il dispositivo espone inoltre un web server locale, raggiungibile via WiFi, che consente all'utente di configurare i parametri di funzionamento, scaricare i dati archiviati in un intervallo temporale specifico e rimuoverli dalla scheda microSD.

3.2 Analisi degli utenti

Prima di affrontare le fasi di progettazione e sviluppo, è opportuno individuare a chi si rivolge il prodotto finale, soprattutto quando questo non è ancora disponibile sul mercato. L'azienda ha già condotto uno studio preliminare del pubblico di riferimento, individuando tre principali categorie di utenti:

- i **centri di *coaching***, che possono impiegare i dati sensoriali raccolti come supporto didattico per il miglioramento tecnico dei piloti.
- i **team organizzatori di eventi motociclistici**, interessati a raccogliere dati telemetrici dettagliati sui partecipanti alle gare;
- i **piloti semi-professionisti**, che possono sfruttare EggLog per valutare le proprie prestazioni durante sessioni di allenamento o competizioni ufficiali;

3.3 Lista degli *use case*

Per facilitare la comprensione dei casi d'uso, questi saranno descritti da uno o più diagrammi ciascuno, che rispettano la sintassi *UML_G*. Ciascun caso d'uso è identificato da un codice seguito dalla lettera **F** o **A**, che ne indica rispettivamente l'appartenenza al **firmware** del dispositivo o all'**applicazione mobile**; i casi d'uso relativi ai due componenti sono presentati in un ordine unico, in modo da mantenere adiacenti quelli tra loro correlati

anche quando appartengono a sistemi diversi. La descrizione testuale deve contenere le seguenti informazioni:

- **Attore Primario:** rappresenta il ruolo che un'entità esterna al sistema assume quando interagisce con esso per raggiungere un obiettivo.
- **Precondizioni:** condizioni che devono essere vere nello stato del sistema prima che il caso d'uso inizi la sua esecuzione.
- **Postcondizioni:** condizioni che devono essere vere nello stato del sistema dopo che il caso d'uso è terminato.
- **Flusso Principale:** interazioni tra attore e sistema che portano al raggiungimento dell'obiettivo del caso d'uso con successo.
- **Flusso Alternativo:** variazioni rispetto al flusso principale.
- **Inclusioni:** riferimenti ai casi d'uso inclusi.
- **Estensioni:** riferimenti ai casi d'uso di estensione.

3.3.1 UC1-F – Acquisizione binaria dei dati sensore

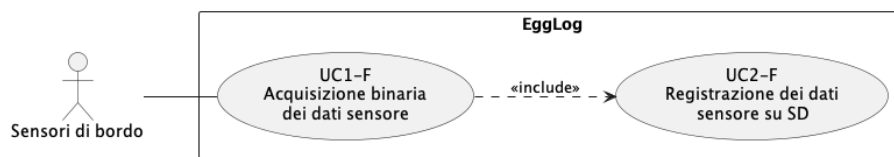


Figura 3.1: UC1-F – Acquisizione binaria dei dati sensore

- **Attore Primario:** sensori di bordo.
- **Precondizioni:**
 - I sensori sono stati inizializzati e, se necessario, calibrati (cfr. [UC16-F](#), [UC20-F](#));
 - è in corso una sessione di registrazione attiva (cfr. [UC15-F](#)).
- **Postcondizioni:** il buffer in RAM contiene i campioni acquisiti, serializzati in formato binario, pronti per essere scritti su SD.
- **Flusso Principale:**
 1. il firmware recupera un nuovo campione da ciascun sensore attivo, secondo la frequenza di campionamento configurata;
 2. ciascun sensore restituisce il valore misurato;
 3. il firmware associa a ogni campione il timestamp corrente;
 4. il firmware serializza il campione in formato binario nativo;
 5. il firmware inserisce il campione serializzato nel buffer in RAM;
 6. se il buffer ha raggiunto la soglia di scrittura, il firmware avvia [UC2-F](#).
- **Inclusioni:** [UC2-F](#).

3.3.2 UC2-F – Registrazione dei dati binari dei sensori su SD

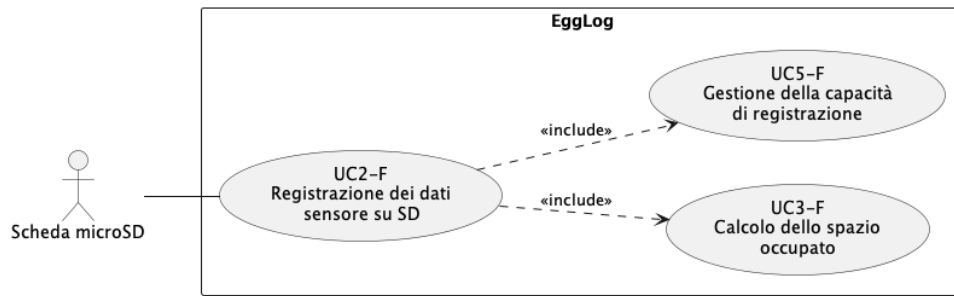


Figura 3.2: Inclusioni UC2-F – Registrazione dei dati sensore su SD

- **Attore Primario:** scheda microSD.
- **Precondizioni:**
 - il buffer di acquisizione (cfr. [UC1-F](#)) contiene dei dati binari pronti per la scrittura;
 - la scheda microSD è presente e correttamente montata;
 - è stata creata la cartella dedicata alla sessione corrente (cfr. [UC21-F](#)), contenente i file destinati a ciascun sensore.
- **Postcondizioni:** i dati sono persistiti nel file dedicato al sensore all'interno della cartella di sessione e il buffer in RAM viene svuotato.
- **Flusso Principale:**
 1. il firmware verifica che il buffer abbia raggiunto la soglia di scrittura;
 2. il firmware accede al file dedicato al sensore all'interno della cartella di sessione corrente;
 3. il firmware scrive il contenuto del buffer in coda al file;
 4. il firmware svuota il buffer in RAM;
 5. il firmware calcola lo spazio occupato dal file (cfr. [UC3-F](#)) e ne verifica la capacità (cfr. [UC5-F](#));
 6. il firmware conferma l'avvenuta scrittura.
- **Flusso Alternativo:**
 - se la scheda microSD non è presente, la scrittura viene rimandata in attesa del reinserimento della scheda, mentre il firmware ne segnala l'assenza via BLE all'applicazione (cfr. [UC8-F](#));
 - se la scrittura fallisce (SD piena o guasta), il firmware segnala l'errore (cfr. [UC12.2-F](#)).
- **Inclusioni:**
 - [UC3-F](#);

– UC5-F.

- **Estensioni:**

– UC8-F;

– UC12.2-F.

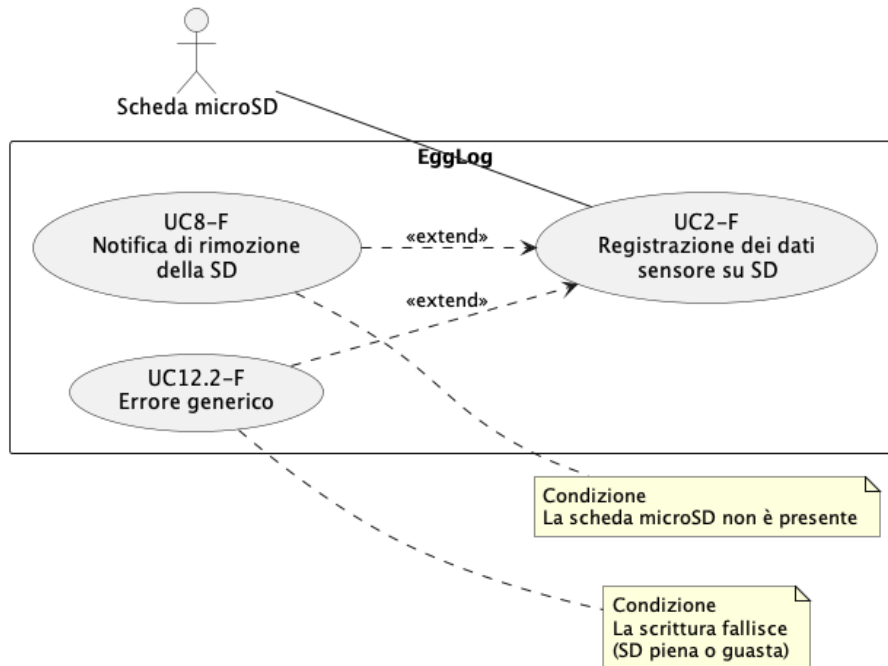


Figura 3.3: UC2-F – Registrazione dei dati sensore su SD

3.3.3 UC3-F – Calcolo dello spazio occupato dai dati registrati

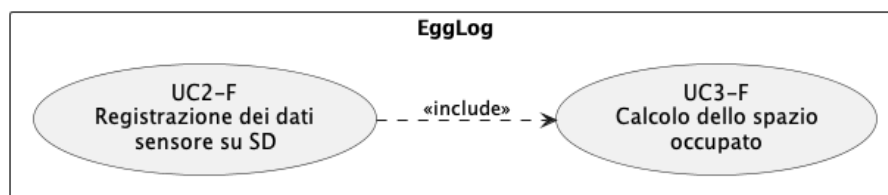


Figura 3.4: UC3-F – Calcolo dello spazio occupato dai dati registrati

- **Attore Primario:** nessuno; il caso d’uso è innescato internamente da UC2-F.
- **Precondizioni:** è stata completata una scrittura di dati sul file dedicato a un sensore.
- **Postcondizioni:** lo spazio occupato dai dati all’interno del file relativo al sensore viene calcolato in modo da poterlo inviare all’applicazione via BLE (cfr. UC4-A) e per la verifica dei limiti di capacità (cfr. UC5-F).
- **Flusso Principale:**

1. il firmware calcola la quantità spazio occupato dai dati scritti nel file relativo al sensore e lo converte in percentuale;
2. il firmware invia all'applicazione la percentuale di spazio occupato associato al sensore.

3.3.4 UC4-A – Visualizzazione dello spazio di archiviazione occupato

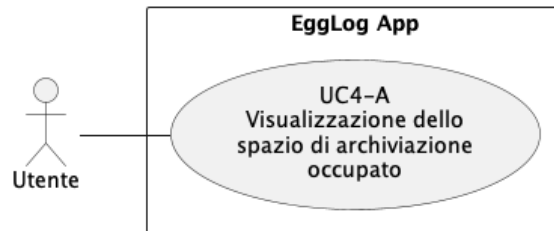


Figura 3.5: UC4-A – Visualizzazione dello spazio di archiviazione occupato

- **Attore Primario:** utente.
- **Precondizioni:**
 - è in corso una sessione di registrazione attiva;
 - il firmware ha calcolato lo spazio occupato per ciascun sensore (cfr. [UC3-F](#)).
- **Postcondizioni:** l'utente visualizza la percentuale di spazio occupato da ogni sensore all'interno del proprio file.
- **Flusso Principale:**
 1. l'applicazione riceve periodicamente lo stato di occupazione dello spazio dal firmware;
 2. l'applicazione effettua un controllo sui limiti delle percentuali di spazio occupato;
 3. l'applicazione mostra all'utente la percentuale di spazio occupato.
- **Flusso Alternativo:**
 - se per un sensore diverso dal GPS lo spazio disponibile ha raggiunto il limite e sono in corso sovrascritture dei dati meno recenti, l'applicazione mostra, sotto la percentuale, l'indicazione "rewriting".

3.3.5 UC5-F – Gestione della capacità di registrazione e blocco automatico

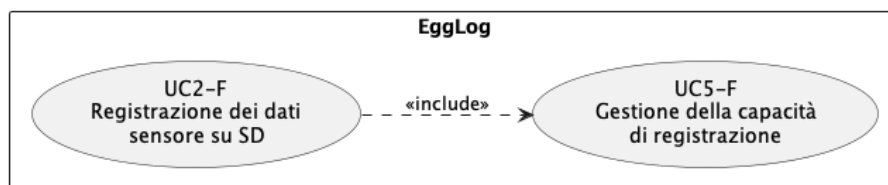


Figura 3.6: UC5-F – Gestione della capacità di registrazione e blocco automatico

- **Attore Primario:** nessuno; il caso d’uso è innescato internamente da [UC2-F](#).
- **Precondizioni:** lo spazio occupato dal file relativo a un sensore è stato calcolato (cfr. [UC3-F](#)).
- **Postcondizioni:** la sessione risulta interrotta oppure i dati meno recenti del sensore risultano sovrascritti, in base al fatto che il GPS abbia o meno raggiunto la capacità massima di archiviazione dentro al proprio file.
- **Flusso Principale:**
 1. il firmware verifica se lo spazio occupato dal file del sensore ha raggiunto il limite previsto per quel sensore;
 2. se il sensore è il modulo GPS, il firmware interrompe automaticamente la sessione di registrazione e lo notifica all’applicazione (cfr. [UC6-A](#));
 3. se il sensore invece è l’IMU, il barometro oppure il magnetometro, il firmware prosegue la registrazione sovrascrivendo i dati meno recenti presenti nel file.

3.3.6 UC6-A – Notifica di sessione bloccata per spazio GPS esaurito

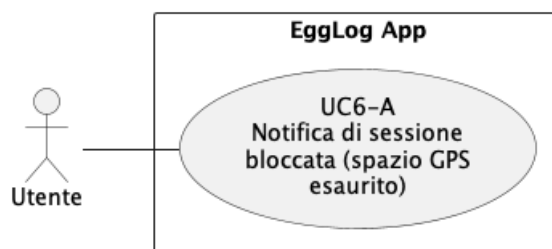


Figura 3.7: UC6-A – Notifica di sessione bloccata per spazio GPS esaurito

- **Attore Primario:** utente.
- **Precondizioni:** il file dedicato al modulo GPS ha raggiunto il limite massimo di spazio previsto (cfr. [UC5-F](#)).
- **Postcondizioni:** la sessione di registrazione risulta interrotta e l’utente è stato informato del motivo dell’interruzione.

- **Flusso Principale:**

1. il firmware notifica all'applicazione l'interruzione della sessione;
2. l'applicazione mostra all'utente un messaggio che indica il raggiungimento della capacità massima prevista per i dati GPS.

3.3.7 UC7-F – Recupero della sessione dopo rimozione SD

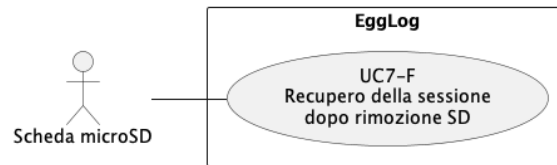


Figura 3.8: UC7-F – Recupero della sessione dopo rimozione SD

- **Attore Primario:** scheda microSD.
- **Precondizioni:** era in corso una sessione di registrazione, interrotta in modo imprevisto a causa della rimozione della scheda microSD.
- **Postcondizioni:** la registrazione riprende dal punto corretto all'interno dei file di sessione, senza perdita né sovrascrittura dei dati già acquisiti.
- **Flusso Principale:**
 1. al reinserimento della scheda microSD, il firmware rileva la presenza di una sessione precedentemente interrotta;
 2. il firmware individua, per ciascun file di sessione, l'ultima posizione di scrittura valida;
 3. il firmware ripristina lo stato di scrittura a partire da tale posizione;
 4. il firmware riprende la registrazione dei dati in coda ai file esistenti.

3.3.8 UC8-F – Notifica di rimozione della scheda microSD

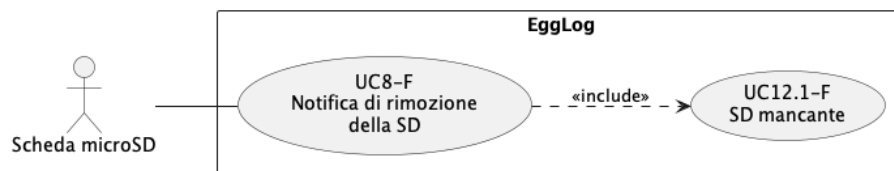


Figura 3.9: UC8-F – Notifica di rimozione della scheda microSD

- **Attore Primario:** scheda microSD.
- **Precondizioni:** è in corso, oppure si sta tentando di avviare, una sessione di registrazione.
- **Postcondizioni:** l'applicazione è stata informata dell'assenza della scheda microSD.

- **Flusso Principale:**

1. il firmware rileva l'assenza della scheda microSD;
2. il firmware notifica lo stato di assenza della scheda all'applicazione;
3. il firmware applica il pattern LED corrispondente (cfr. [UC12.1-F](#)).

- **Inclusioni:** [UC12.1-F](#).

3.3.9 UC9-A – Gestione della rimozione SD durante la sessione

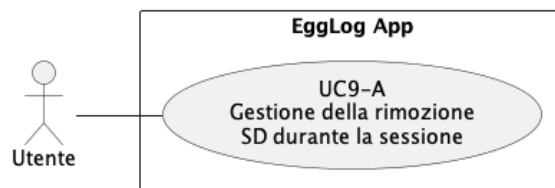


Figura 3.10: UC9-A – Gestione della rimozione SD durante la sessione

- **Attore Primario:** utente.

- **Precondizioni:** è in corso una sessione di registrazione attiva e il firmware ha notificato l'assenza della scheda microSD (cfr. [UC8-F](#)).

- **Postcondizioni:** la sessione risulta sospesa oppure interrotta, in base alla scelta dell'utente o al successivo reinserimento della scheda.

- **Flusso Principale:**

1. l'applicazione riceve la notifica di assenza della scheda microSD;
2. l'applicazione sospende la sessione e mostra il messaggio di scheda microSD mancante, offrendo la possibilità di interromperla manualmente;
3. l'utente reinserisce la scheda microSD;
4. l'applicazione rileva il ripristino della scheda, chiude il messaggio e la sessione riprende normalmente.

- **Flusso Alternativo:**

- se l'utente preme il tasto per interrompere la sessione, la sessione viene terminata definitivamente.

3.3.10 UC10-A – Gestione dell'assenza della SD all'avvio sessione

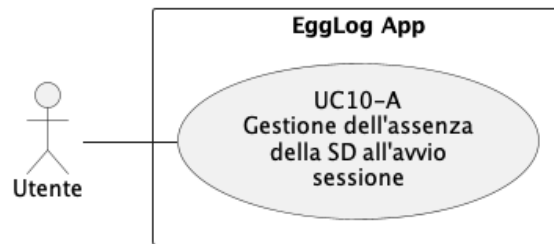


Figura 3.11: UC10-A – Gestione dell'assenza della SD all'avvio sessione

- **Attore Primario:** utente.
- **Precondizioni:** l'utente ha richiesto l'avvio di una nuova sessione di registrazione e il firmware ha notificato l'assenza della scheda microSD (cfr. [UC8-F](#)).
- **Postcondizioni:** la sessione viene avviata solo dopo che la scheda microSD risulta correttamente inserita.
- **Flusso Principale:**
 1. l'utente richiede l'avvio di una nuova sessione;
 2. l'applicazione riceve la notifica di assenza della scheda microSD;
 3. l'applicazione mostra il messaggio di scheda microSD mancante con il pulsante per ritentare l'avvio della sessione;
 4. l'utente inserisce la scheda microSD e ritenta l'avvio della sessione;
 5. il firmware verifica la presenza della scheda e, in parallelo con l'applicazione, avvia la sessione (cfr. [UC15-F](#)).
- **Flusso Alternativo:**
 - se la scheda risulta ancora assente, l'applicazione mostra nuovamente il messaggio di scheda microSD mancante con il pulsante per ritentare l'avvio della sessione.

3.3.11 UC11-F – Acquisizione dati dal bus CAN

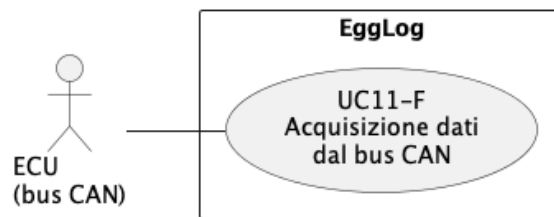


Figura 3.12: UC11-F – Acquisizione dati dal CAN

- **Attore Primario:** ECU del veicolo mediante bus CAN.

- **Precondizioni:**

- il modulo di interfaccia CAN è collegato al bus dati del veicolo;
- è in corso una sessione di registrazione attiva.

- **Postcondizioni:** i dati ECU acquisiti sono associati ad un timestamp e resi disponibili per la registrazione (cfr. [UC2-F](#)).

- **Flusso Principale:**

1. il firmware si mette in ascolto sul bus CAN;
2. la ECU trasmette periodicamente i messaggi contenenti i parametri motore;
3. il firmware riceve e decodifica ciascun messaggio CAN;
4. il firmware associa a ciascun dato il timestamp corrente;
5. il firmware serializza i dati in formato binario;;
6. il firmware inserisce i dati nel buffer di acquisizione.

- **Flusso Alternativo:**

- se il messaggio CAN ricevuto non è riconosciuto o risulta malformato, il messaggio viene scartato.

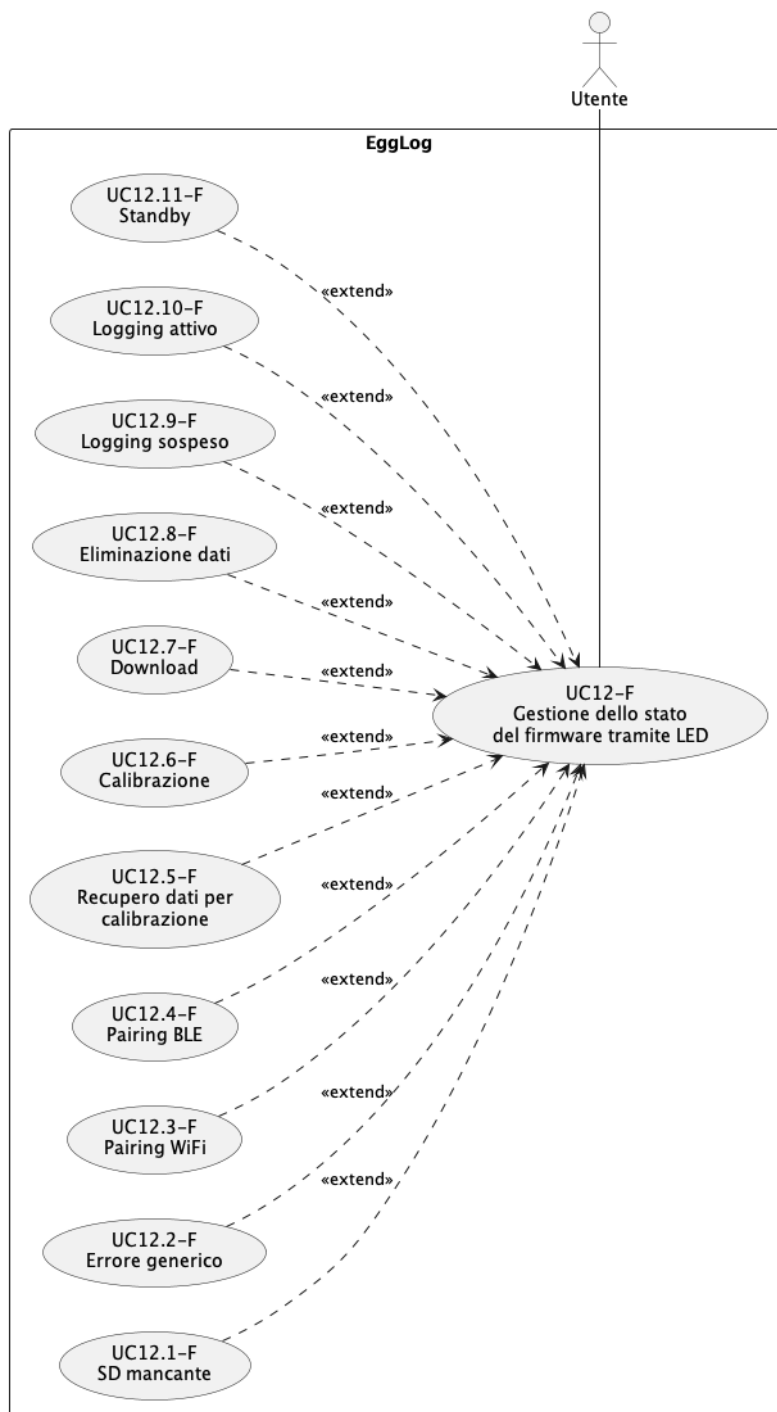
3.3.12 UC12-F – Gestione dello stato del firmware tramite *LED_G*

Figura 3.13: UC12-F – Gestione dello stato del firmware tramite LED

- **Attore Primario:** utente.
- **Precondizioni:** il dispositivo è alimentato e il firmware è in esecuzione.
- **Postcondizioni:** il LED riflette lo stato operativo corrente del dispositivo, secondo un ordine di priorità tra le condizioni concorrenti.
- **Flusso Principale:**

1. il firmware rileva un cambiamento nello stato operativo del dispositivo;
 2. il firmware seleziona il pattern di colore/lampeggio associato alla condizione a priorità più alta tra quelle attive;
 3. il firmware applica il pattern al LED;
 4. l'utente osserva il LED e ne interpreta lo stato del dispositivo.
- **Flusso Alternativo:**
 - se si verificano più condizioni contemporaneamente, il firmware seleziona il pattern relativo alla condizione a priorità più alta.
 - **Estensioni:**
 - UC12.1-F;
 - UC12.2-F;
 - UC12.3-F;
 - UC12.4-F;
 - UC12.5-F;
 - UC12.6-F;
 - UC12.7-F;
 - UC12.8-F;
 - UC12.9-F;
 - UC12.10-F;
 - UC12.11-F.

3.3.12.1 UC12.1-F – Segnalazione di scheda microSD mancante

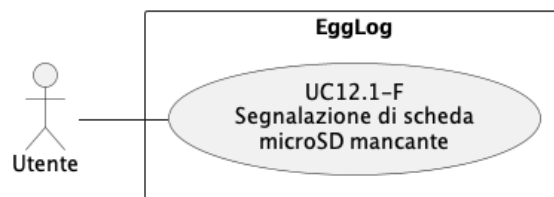
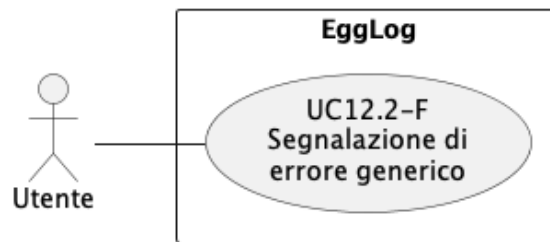
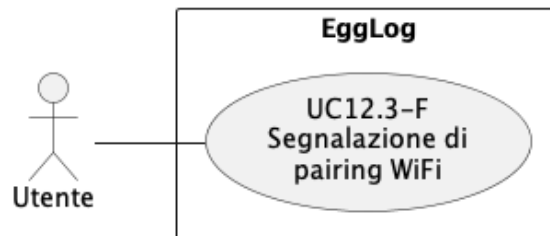


Figura 3.14: UC12.1-F – Segnalazione di scheda microSD mancante

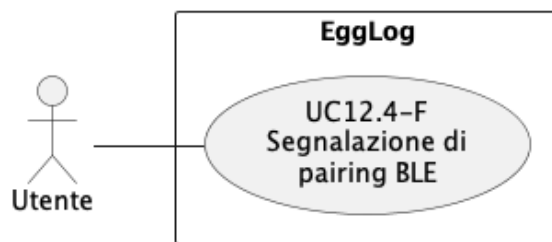
- **Attore Primario:** utente.
- **Precondizioni:** la scheda microSD risulta assente (cfr. UC8-F) e si sta tentando di avviare una sessione oppure si è in piena sessione di registrazione.
- **Postcondizioni:** il LED mostra il pattern rosso con lampeggio veloce, con priorità massima su ogni altro stato.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'assenza della scheda microSD;
 2. il modulo applica il pattern rosso con lampeggio veloce;
 3. il pattern resta attivo finché la scheda non viene rilevata correttamente.

3.3.12.2 UC12.2-F – Segnalazione di errore generico**Figura 3.15:** UC12.2-F – Segnalazione di errore generico

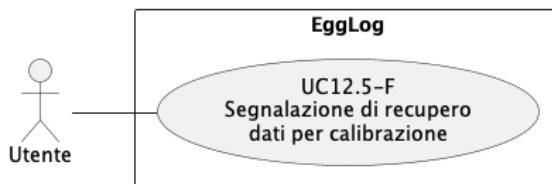
- **Attore Primario:** utente.
- **Precondizioni:** si verifica una condizione di errore o malfunzionamento generica.
- **Postcondizioni:** il LED mostra il pattern rosso fisso, con priorità su ogni stato successivo.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva la condizione di errore;
 2. il modulo applica il pattern rosso fisso;
 3. il pattern resta attivo finché l'errore non viene risolto o il dispositivo riavviato.

3.3.12.3 UC12.3-F – Segnalazione di pairing WiFi**Figura 3.16:** UC12.3-F – Segnalazione di pairing WiFi

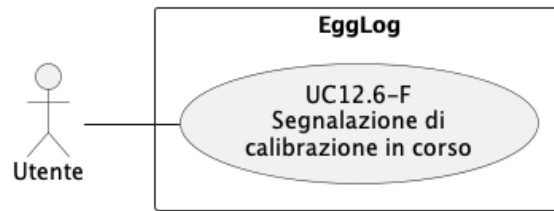
- **Attore Primario:** utente.
- **Precondizioni:** è in corso una procedura di associazione *pairing* con una rete WiFi.
- **Postcondizioni:** il LED mostra il pattern blu con lampeggio veloce.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio della procedura di pairing WiFi;
 2. il modulo applica il pattern blu con lampeggio veloce;
 3. il pattern rimane attivo per l'intera durata del pairing.

3.3.12.4 UC12.4-F – Segnalazione di pairing BLE**Figura 3.17:** UC12.4-F – Segnalazione di pairing BLE

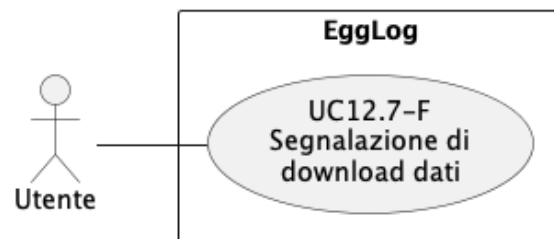
- **Attore Primario:** utente.
- **Precondizioni:** è in corso una procedura di associazione (*pairing*) con un dispositivo tramite BLE.
- **Postcondizioni:** il LED mostra il pattern blu con lampeggio lento.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio della procedura di pairing BLE;
 2. il modulo applica il pattern blu con lampeggio lento;
 3. il pattern rimane attivo per l'intera durata del pairing.

3.3.12.5 UC12.5-F – Segnalazione di recupero dati per calibrazione**Figura 3.18:** UC12.5-F – Segnalazione di recupero dati per calibrazione

- **Attore Primario:** utente.
- **Precondizioni:** è in corso l'acquisizione dei campioni necessari a una procedura di calibrazione (cfr. [UC16-F](#)).
- **Postcondizioni:** il LED mostra il pattern giallo fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio dell'acquisizione dei campioni di calibrazione;
 2. il modulo applica il pattern giallo fisso;
 3. il pattern rimane attivo per l'intera durata della fase di acquisizione dei dati di calibrazione.

3.3.12.6 UC12.6-F – Segnalazione di calibrazione in corso**Figura 3.19:** UC12.6-F – Segnalazione di calibrazione in corso

- **Attore Primario:** utente.
- **Precondizioni:** è in corso l’elaborazione dei dati di calibrazione.
- **Postcondizioni:** il LED mostra il pattern giallo pulsante (effetto *breathing*).
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l’avvio dell’elaborazione dei dati di calibrazione;
 2. il modulo applica il pattern giallo pulsante;
 3. il pattern rimane attivo finché la calibrazione non è completata.

3.3.12.7 UC12.7-F – Segnalazione di download dati**Figura 3.20:** UC12.7-F – Segnalazione di download dati

- **Attore Primario:** utente.
- **Precondizioni:** è in corso il download dei dati registrati verso l’applicazione.
- **Postcondizioni:** il LED mostra il pattern viola fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l’avvio del download dei dati;
 2. il modulo applica il pattern viola fisso;
 3. il pattern rimane attivo per l’intera durata del download.

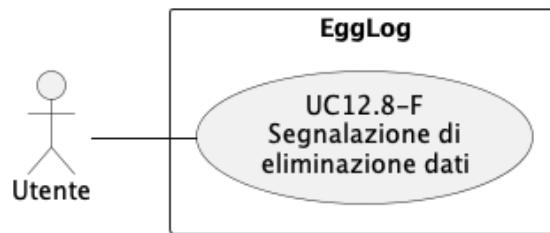
3.3.12.8 UC12.8-F – Segnalazione di eliminazione dati

Figura 3.21: UC12.8-F – Segnalazione di eliminazione dati

- **Attore Primario:** utente.
- **Precondizioni:** è in corso l’eliminazione dei dati registrati dalla scheda microSD.
- **Postcondizioni:** il LED mostra il pattern viola con lampeggio regolare.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l’avvio dell’eliminazione dei dati;
 2. il modulo applica il pattern viola con lampeggio regolare;
 3. il pattern rimane attivo per l’intera durata dell’eliminazione.

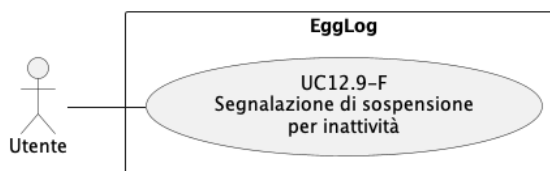
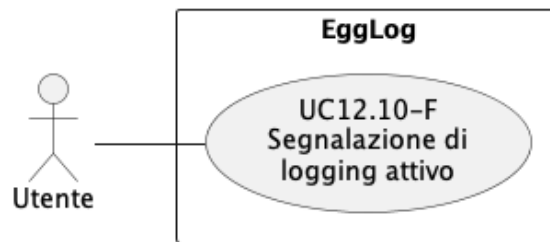
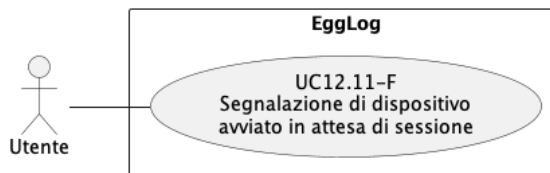
3.3.12.9 UC12.9-F – Segnalazione di sospensione per inattività

Figura 3.22: UC12.9-F – Segnalazione di sospensione per inattività

- **Attore Primario:** utente.
- **Precondizioni:** si verifica la sospensione automatica dell’acquisizione sensori descritta in [UC13-F](#).
- **Postcondizioni:** il LED mostra il pattern verde pulsante lento (effetto *breathing*).
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva la sospensione dell’acquisizione sensori;
 2. il modulo applica il pattern verde pulsante lento;
 3. il pattern rimane attivo finché l’acquisizione sensori resta sospesa.

3.3.12.10 UC12.10-F – Segnalazione di logging attivo**Figura 3.23:** UC12.10-F – Segnalazione di logging attivo

- **Attore Primario:** utente.
- **Precondizioni:** è in corso una regolare sessione di registrazione dati (cfr. [UC15-F](#)).
- **Postcondizioni:** il LED mostra il pattern verde fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'avvio, o la ripresa, della sessione di registrazione;
 2. il modulo applica il pattern verde fisso;
 3. il pattern rimane attivo per l'intera durata della sessione.

3.3.12.11 UC12.11-F – Segnalazione di dispositivo avviato in attesa di sessione**Figura 3.24:** UC12.11-F – Segnalazione di dispositivo avviato in attesa di sessione

- **Attore Primario:** utente.
- **Precondizioni:** il dispositivo è stato avviato correttamente, oppure una sessione precedente è appena terminata (cfr. [UC15-F](#)), e non è attiva alcuna delle altre condizioni.
- **Postcondizioni:** il LED mostra il pattern bianco fisso.
- **Flusso Principale:**
 1. il modulo di gestione del LED rileva l'assenza di ogni altra condizione a priorità più alta;
 2. il modulo applica il pattern bianco fisso;
 3. il pattern resta attivo finché non si presenta una condizione a priorità più alta.

3.3.13 UC13-F – Sospensione dell’acquisizione sensori in stato di inattività

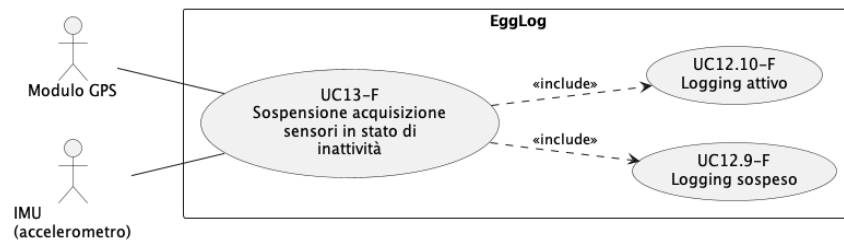


Figura 3.25: UC13-F – Sospensione dell’acquisizione sensori in stato di inattività

- **Attore Primario:** modulo GPS
- **Attore Secondario** IMU (accelerometro).
- **Precondizioni:** è in corso una sessione di registrazione attiva.
- **Postcondizioni:** l’acquisizione dei sensori non essenziali risulta sospesa o riattivata in base allo stato di movimento rilevato.
- **Flusso Principale:**
 1. se il modulo GPS dispone di un *fix* valido, il firmware monitora la velocità rilevata dal GPS; in caso contrario monitora i dati dell’accelerometro dell’IMU;
 2. il firmware verifica se non si sono registrate variazioni significative per l’intervallo di tempo configurato;
 3. al superamento dell’intervallo, il firmware sospende l’acquisizione dei sensori non essenziali;
 4. il firmware applica il pattern LED di sospensione (cfr. [UC12.9-F](#));
 5. il firmware continua a monitorare la sorgente disponibile (GPS o IMU) per rilevare un nuovo movimento;
 6. al rilevamento di movimento, il firmware riattiva l’acquisizione e applica [UC12.10-F](#) per ripristinare il pattern di logging attivo.
- **Flusso Alternativo:**
 - se la sospensione automatica è stata disabilitata dall’utente (cfr. [UC14-A](#)), il firmware non sospende mai l’acquisizione, indipendentemente dallo stato di movimento rilevato.
- **Inclusioni:**
 - [UC12.9-F](#);
 - [UC12.10-F](#).

3.3.14 UC14-A – Configurazione del tempo di inattività

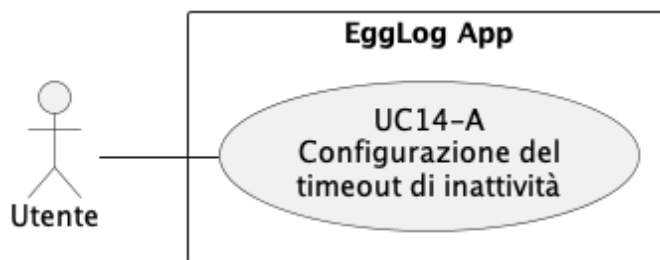


Figura 3.26: UC14-A – Configurazione del tempo di inattività

- **Attore Primario:** utente.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** il valore configurato per il tempo di inattività è salvato e applicato alla sospensione automatica dell'acquisizione (cfr. [UC13-F](#)).
- **Flusso Principale:**
 1. l'utente seleziona uno dei valori disponibili per il tempo di inattività;
 2. l'applicazione trasmette il valore selezionato al firmware;
 3. il firmware salva la nuova configurazione e la applica alle sessioni successive.

3.3.15 UC15-F – Avvio e interruzione sessione tramite comandi BLE

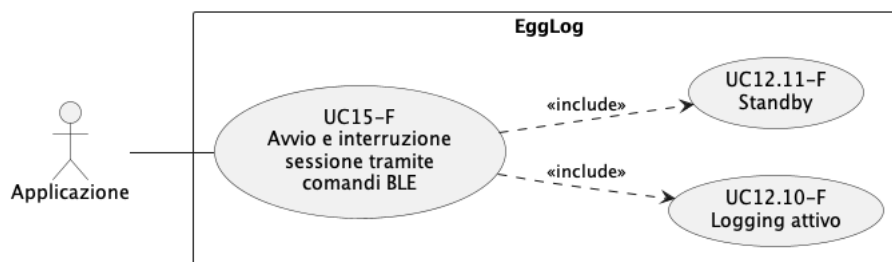


Figura 3.27: UC15-F – Avvio e interruzione sessione tramite comandi BLE

- **Attore Primario:** applicazione.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** la sessione di registrazione risulta avviata o interrotta in base al comando ricevuto; i sensori non essenziali restano dormienti al di fuori di una sessione attiva.
- **Flusso Principale:**
 1. il firmware riceve, tramite BLE, un comando di avvio o interruzione sessione;

2. se il comando è di avvio, il firmware attiva i sensori, crea la cartella di sessione (cfr. UC21-F), esegue le calibrazioni automatiche previste (cfr. UC20-F) e applica il pattern LED di logging attivo (cfr. UC12.10-F);
 3. se il comando è di interruzione, il firmware termina la registrazione, pone i sensori non essenziali in stato dormiente e applica il pattern LED di attesa sessione (cfr. UC12.11-F).
- **Inclusioni:**
 - UC12.10-F;
 - UC12.11-F.

3.3.16 UC16-F – Calibrazione dell'IMU

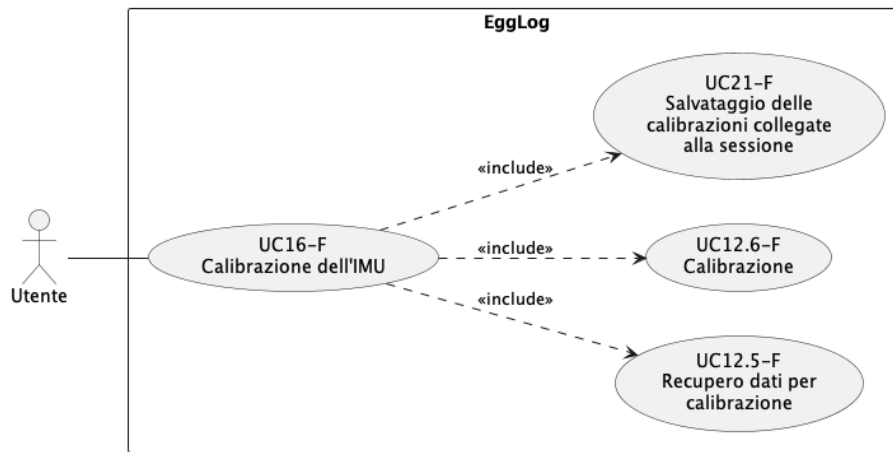


Figura 3.28: UC16-F – Calibrazione dell'IMU

- **Attore Primario:** utente.
- **Precondizioni:** è stabilita una connessione BLE tra dispositivo e applicazione.
- **Postcondizioni:** i parametri di calibrazione, calcolati a partire dai dati di giroscopio e accelerometro dell'IMU, sono salvati (cfr. UC21-F) e applicati alle letture sensore successive, indipendentemente dal posizionamento del dispositivo sul veicolo.
- **Flusso Principale:**
 1. il firmware avvia la procedura di calibrazione, guidando l'utente attraverso una sequenza di movimenti (cfr. UC17-A);
 2. per ciascun movimento, il firmware acquisisce i campioni di giroscopio e accelerometro, applicando il pattern LED corrispondente (cfr. UC12.5-F);
 3. il firmware elabora i dati raccolti applicando il pattern LED corrispondente (cfr. UC12.6-F);
 4. il firmware applica la matrice di calibrazione e i dati di calibrazione alle acquisizioni successive (cfr. UC1-F).

- **Inclusioni:**
 - UC12.5-F;
 - UC12.6-F;
 - UC21-F.

3.3.17 UC17-A – Sequenza guidata di calibrazione dell'IMU

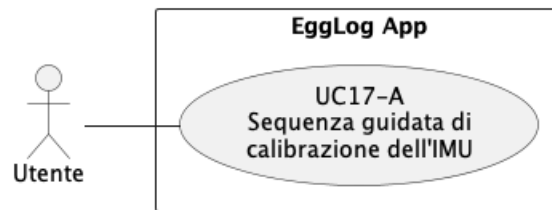


Figura 3.29: UC17-A – Sequenza guidata di calibrazione dell'IMU

- **Attore Primario:** utente.
- **Precondizioni:** è stata avviata una procedura di calibrazione dell'IMU (cfr. UC16-F).
- **Postcondizioni:** l'utente ha completato correttamente i tre movimenti richiesti dalla procedura di calibrazione.
- **Flusso Principale:**
 1. l'applicazione richiede all'utente di tenere la moto ferma e perfettamente in piedi;
 2. l'applicazione richiede all'utente di inclinare la moto verso destra;
 3. l'applicazione richiede all'utente di camminare per 2-3 metri in avanti con la moto;
 4. al termine dei tre passaggi, l'applicazione mostra l'esito della calibrazione.
- **Flusso Alternativo:**
 - se i dati raccolti durante uno dei tre passaggi risultano poco significativi o non conformi rispetto al passaggio precedente, l'applicazione mostra un messaggio che richiede la ripetizione del movimento corrente.

3.3.18 UC18-A – Visualizzazione dei dati calibrati

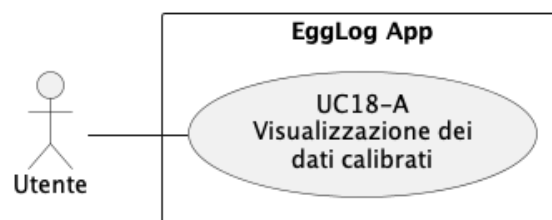


Figura 3.30: UC18-A – Visualizzazione dei dati calibrati

- **Attore Primario:** utente.
- **Precondizioni:**
 - l'utente ha avviato una sessione;
 - sono state completate le calibrazioni previste per la sessione (cfr. [UC16-F](#), [UC20-F](#)).
- **Postcondizioni:** l'utente visualizza i dati sensore corretti secondo i parametri di calibrazione applicati.
- **Flusso Principale:**
 1. il firmware trasmette all'applicazione i dati sensore già calibrati (cfr. [UC21-F](#));
 2. l'applicazione mostra i valori corretti all'utente.

3.3.19 UC19-A – Visualizzazione dei movimenti della moto in tempo reale

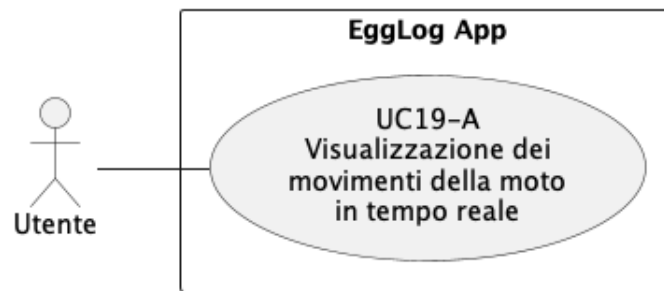


Figura 3.31: UC19-A – Visualizzazione dei movimenti della moto in tempo reale

- **Attore Primario:** utente.
- **Precondizioni:** è in corso una sessione di registrazione con calibrazione IMU completata (cfr. [UC16-F](#)).
- **Postcondizioni:** l'utente visualizza in tempo reale i movimenti della moto rilevati dall'IMU.
- **Flusso Principale:**
 1. il firmware trasmette in tempo reale all'applicazione i dati calibrati di inclinazione frontale, inclinazione laterale e accelerazione frontale;
 2. l'applicazione mostra tali valori numerici all'utente;
 3. l'applicazione applica gli stessi dati a un modello tridimensionale della moto, replicando i movimenti sui rispettivi assi X, Y e Z.

3.3.20 UC20-F – Calibrazione automatica dell'altitudine a inizio sessione

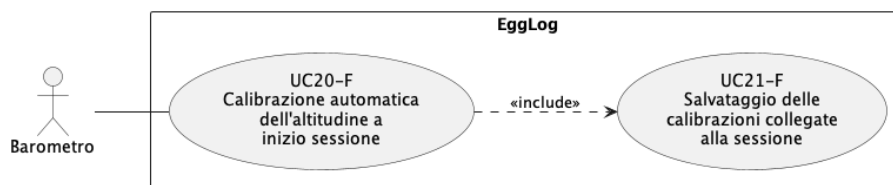


Figura 3.32: UC20-F – Calibrazione automatica dell'altitudine a inizio sessione

- **Attore Primario:** barometro.
- **Precondizioni:** è stata avviata una nuova sessione di registrazione (cfr. [UC15-F](#)).
- **Postcondizioni:** il valore di altitudine iniziale, calcolato tramite il barometro, è salvato e associato alla sessione corrente (cfr. [UC21-F](#)).
- **Flusso Principale:**
 1. all'avvio della sessione, il firmware acquisisce le letture del barometro;
 2. il firmware calcola il valore di altitudine di riferimento;
 3. il firmware applica tale valore come riferimento alle letture di altitudine successive.
- **Flusso Alternativo:** i dati raccolti dal barometro non risultano abbastanza stabili per cui la sessione si avvia senza una calibrazione dell'altitudine di partenza.
- **Inclusioni:** [UC21-F](#).

3.3.21 UC21-F – Salvataggio delle calibrazioni collegate alla sessione

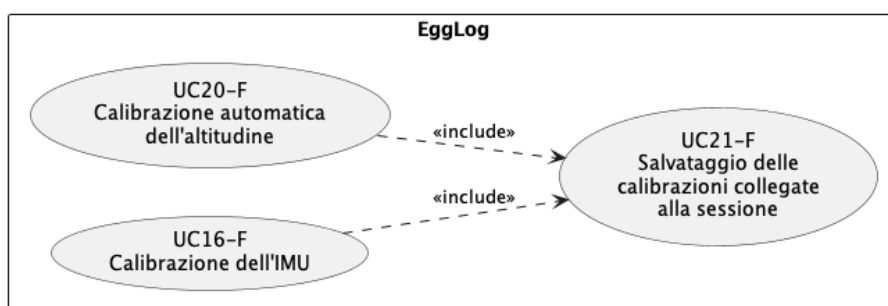


Figura 3.33: UC21-F – Salvataggio delle calibrazioni collegate alla sessione

- **Attore Primario:** nessuno; il caso d'uso è innescato internamente da [UC16-F](#) e da [UC20-F](#).
- **Precondizioni:** è stata completata una procedura di calibrazione (IMU o altitudine).

- **Postcondizioni:** i dati di calibrazione dell'altitudine sono salvati all'interno della cartella di sessione, mentre i dati di calibrazione dell'IMU sono salvati nella cartella principale contenente tutte le cartelle di sessione.
- **Flusso Principale:**
 1. il firmware crea, se non già presenti, la cartella dedicata al salvataggio delle sessioni e la cartella della sessione corrente;
 2. il firmware salva il file di calibrazione dell'altitudine all'interno della cartella di sessione, mentre salva il file di calibrazione dell'IMU nella cartella principale.

3.4 Tracciamento dei requisiti

In questa sezione si riporta la lista di requisiti rilevata in seguito all'analisi dell'utenza e degli use case; i requisiti sono identificati da un codice definito con la seguente nomenclatura:

R-(Tipo)-(Urgenza)-(Codice)

Dove:

- **Tipo:** F (Funzionale), N (Non funzionale) e V (Vincolo);
- **Urgenza:** Ob (Obbligatoria), De (Desiderabile) e Op (Opzionale);
- **Codice:** numero progressivo del requisito.

3.4.1 Requisiti funzionali

Nella Tabella 3.1 si riporta il tracciamento dei requisiti funzionali, ovvero le funzionalità e i servizi che il sistema deve offrire per essere conforme alle aspettative.

Codice	Descrizione	Fonti
R-F-Ob-1	EggLog deve acquisire e serializzare in formato binario i dati provenienti dai sensori	UC1-F
R-F-Ob-2	EggLog deve scrivere su scheda microSD i dati acquisiti in formato binario	UC2-F
R-F-Ob-3	EggLog deve calcolare e trasmettere all'applicazione, per ciascun sensore, la percentuale di spazio occupato nel relativo file di sessione	UC3-F
R-F-De-4	L'utente deve poter visualizzare in EggLog App la percentuale di spazio di archiviazione occupato da ciascun sensore, incluso lo stato di sovrascrittura in corso	UC4-A
Continua nella prossima pagina...		

Tabella 3.1 – Continuo della tabella

Codice	Descrizione	Fonti
R-F-Ob-5	EggLog deve interrompere automaticamente la sessione al raggiungimento della capacità massima dei dati GPS, mentre deve proseguire la registrazione sovrascrivendo i dati meno recenti per gli altri sensori	UC5-F
R-F-Ob-6	L'utente deve essere informato in EggLog App quando la sessione viene interrotta per esaurimento dello spazio dedicato ai dati GPS	UC6-A
R-F-Ob-7	EggLog deve poter riprendere una sessione interrotta in modo imprevisto (rimozione della scheda microSD oppure un guasto al modulo SD) senza perdita né sovrascrittura dei dati già acquisiti	UC7-F
R-F-Ob-8	EggLog deve rilevare e notificare a EggLog App l'assenza della scheda microSD	UC8-F
R-F-De-9	EggLog App deve sospendere la visualizzazione della sessione in corso quando la scheda microSD viene rimossa, offrendo la possibilità di interromperla manualmente o di riprenderla al reinserimento	UC9-A
R-F-De-10	EggLog App deve impedire l'avvio di una nuova sessione in assenza della scheda microSD, offrendo la possibilità di riprovare dopo il suo inserimento	UC10-A
R-F-Ob-11	EggLog deve leggere i dati esposti dalla centralina motore (ECU) tramite bus CAN	UC11-F
R-F-De-12	EggLog deve pilotare il LED integrato sulla scheda ESP32-S3 per segnalare, tramite pattern di colore e lampeggio distinti e reciprocamente esclusivi, lo stato corrente del dispositivo	UC12-F, UC12.1-F, UC12.2-F, UC12.3-F, UC12.4-F, UC12.5-F, UC12.6-F, UC12.7-F, UC12.8-F, UC12.9-F, UC12.10-F, UC12.11-F
R-F-De-13	EggLog deve sospendere automaticamente l'acquisizione dei sensori non essenziali in stato di inattività prolungata e riattivarla al rilevamento di movimento	UC13-F
Continua nella prossima pagina...		

Tabella 3.1 – Continuo della tabella

Codice	Descrizione	Fonti
R-F-Op-14	L'utente deve poter configurare da EggLog App il tempo di inattività che determina la sospensione automatica dei sensori (mai, 10, 30 o 60 minuti)	UC14-A
R-F-Ob-15	EggLog deve avviare e interrompere la sessione di registrazione in seguito a comandi BLE ricevuti da EggLog App	UC15-F
R-F-Ob-16	EggLog deve calibrare l'IMU guidando l'utente, tramite EggLog App, attraverso una sequenza di movimenti predefinita, applicando poi tali calibrazioni alle letture successive	UC16-F, UC17-A
R-F-De-17	L'utente deve poter visualizzare in EggLog App i dati sensore corretti secondo i parametri di calibrazione applicati	UC18-A
R-F-De-18	L'utente deve poter visualizzare in tempo reale, tramite un modello tridimensionale in EggLog App, i movimenti della moto rilevati dall'IMU (inclinazione frontale, inclinazione laterale e accelerazione frontale)	UC19-A
R-F-Ob-19	EggLog deve calcolare automaticamente, tramite il barometro, un valore di altitudine di riferimento a inizio sessione e applicarlo alle letture successive	UC20-F
R-F-Ob-20	EggLog deve salvare i dati di calibrazione dell'IMU nella cartella principale delle sessioni e i dati di calibrazione dell'altitudine all'interno della cartella della sessione corrente	UC21-F

Tabella 3.1: Tracciamento dei requisiti funzionali.

3.4.2 Requisiti non funzionali

Nella Tabella 3.2 si riporta il tracciamento dei requisiti non funzionali, ovvero l'insieme di criteri quantificabili di qualità che il sistema sviluppato deve rispettare per essere conforme alle aspettative e agli obiettivi preposti.

Codice	Descrizione	Fonti
R-N-Ob-1	Il consumo di RAM per il buffering dei dati sensore deve essere ridotto rispetto alla configurazione precedente del firmware	UC1-F
R-N-Ob-2	Il consumo di CPU del dispositivo deve essere ridotto rispetto alla configurazione precedente	UC1-F, UC13-F
Continua nella prossima pagina...		

Tabella 3.2 – Continuo della tabella

Codice	Descrizione	Fonti
R-N-Ob-3	Il ripristino di una sessione interrotta in modo imprevisto non deve comportare perdita né duplicazione dei dati già acquisiti	UC7-F
R-N-De-4	Il cambio di stato segnalato dal LED deve essere percepito come pressoché istantaneo dall'utente, e ogni pattern di colore/lampeggio deve risultare univocamente distinguibile dagli altri	UC12-F, UC12.1-F, UC12.2-F, UC12.3-F, UC12.4-F, UC12.5-F, UC12.6-F, UC12.7-F, UC12.8-F, UC12.9-F, UC12.10-F, UC12.11-F
R-N-De-5	L'accuratezza delle misurazioni successive alla calibrazione deve rientrare entro le tolleranze specifiche	UC16-F, UC20-F
R-N-De-6	Le notifiche relative a eventi critici (assenza scheda microSD, esaurimento spazio GPS) devono essere recapitate a EggLog App entro un tempo breve dal loro verificarsi	UC8-F, UC6-A, UC9-A, UC10-A
R-N-Op-7	L'aggiornamento dei dati di movimento mostrati sul modello tridimensionale in tempo reale deve risultare fluido e privo di percepibili ritardi	UC19-A

Tabella 3.2: Tracciamento dei requisiti non funzionali.

3.4.3 Requisiti di vincolo

Nella Tabella 3.3 si riporta il tracciamento dei requisiti di vincolo, ovvero l'insieme delle limitazioni tecnologico-organizzative a cui il dispositivo deve sottostare per essere conforme alle aspettative e alle esigenze degli stakeholder.

Codice	Descrizione	Fonti
R-V-Ob-1	Il microcontrollore del dispositivo deve appartenere alla famiglia ESP32, nello specifico ESP32-S3	Tutti
R-V-Ob-2	Il firmware del dispositivo dev'essere scritto in linguaggio C/C++	Tutti
Continua nella prossima pagina...		

Tabella 3.3 – Continuo della tabella

Codice	Descrizione	Fonti
R-V-Ob-3	L'interfaccia con il bus CAN del veicolo deve essere compatibile con il protocollo CAN utilizzato dalla centralina (ECU) della moto	UC11-F
R-V-Ob-4	Il LED utilizzato per la segnalazione di stato deve essere quello integrato sulla scheda ESP32-S3	UC12-F
R-V-Ob-5	La comunicazione tra EggLog ed EggLog App per la configurazione, il controllo della sessione e lo scambio dati in tempo reale deve avvenire tramite BLE	UC14-A, UC15-F, UC16-F

Tabella 3.3: Tracciamento dei requisiti di vincolo.

3.4.4 Tracciamento Fonti – Requisiti

Nella Tabella 3.4 si riporta, per ciascun caso d'uso, l'elenco dei requisiti da esso derivati, a complemento del tracciamento inverso riportato nelle tabelle precedenti.

Fonte	Requisiti
UC1-F	R-F-Ob-1, R-N-Ob-1, R-N-Ob-2
UC2-F	R-F-Ob-2
UC3-F	R-F-Ob-3
UC4-A	R-F-De-4
UC5-F	R-F-Ob-5
UC6-A	R-F-Ob-6, R-N-De-6
UC7-F	R-F-Ob-7, R-N-Ob-3
UC8-F	R-F-Ob-8, R-N-De-6
UC9-A	R-F-De-9, R-N-De-6
UC10-A	R-F-De-10, R-N-De-6
UC11-F	R-F-Ob-11, R-V-Ob-3
UC12-F	R-F-De-12, R-N-De-4, R-V-Ob-4
UC12.1-F	R-F-De-12, R-N-De-4
UC12.2-F	R-F-De-12, R-N-De-4
UC12.3-F	R-F-De-12, R-N-De-4
UC12.4-F	R-F-De-12, R-N-De-4
UC12.5-F	R-F-De-12, R-N-De-4
UC12.6-F	R-F-De-12, R-N-De-4
UC12.7-F	R-F-De-12, R-N-De-4
UC12.8-F	R-F-De-12, R-N-De-4
UC12.9-F	R-F-De-12, R-N-De-4
Continua nella prossima pagina...	

Tabella 3.4 – Continuo della tabella

Fonte	Requisiti
UC12.10-F	R-F-De-12, R-N-De-4
UC12.11-F	R-F-De-12, R-N-De-4
UC13-F	R-F-De-13, R-N-Ob-2
UC14-A	R-F-Op-14, R-V-Ob-5
UC15-F	R-F-Ob-15, R-V-Ob-5
UC16-F	R-F-Ob-16, R-N-De-5, R-V-Ob-5
UC17-A	R-F-Ob-16
UC18-A	R-F-De-17
UC19-A	R-F-De-18, R-N-Op-7
UC20-F	R-F-Ob-19, R-N-De-5
UC21-F	R-F-Ob-20
Tutti	R-V-Ob-1, R-V-Ob-2

Tabella 3.4: Tracciamento Fonti – Requisiti.

3.4.5 Riepilogo dei requisiti

Nella Tabella 3.5 si riporta il numero di requisiti per categoria e per urgenza; in totale sono stati rilevati 32 requisiti.

	Obbligatoria	Desiderabili	Opzionali	TOTALE
RF	12	7	1	20
RN	3	3	1	7
RV	5	0	0	5
TOTALE	20	10	2	32

Tabella 3.5: Conteggio dei requisiti tracciati.

4. Tecnologie utilizzate

In questa sezione vengono presentate le principali tecnologie impiegate durante lo sviluppo del progetto. La descrizione ha lo scopo di fornire il contesto necessario alla comprensione delle scelte implementative illustrate nei capitoli successivi.

4.1 ESP32-S3-DevKitC-1

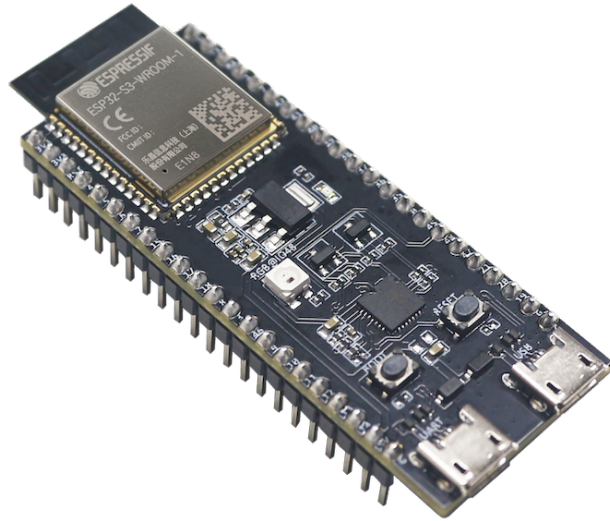


Figura 4.1: Scheda di sviluppo ESP32-S3-DevKitC-1

La *ESP32-S3-DevKitC-1* è una scheda di sviluppo basata sul microcontrollore ESP32-S3, prodotto da Espressif Systems. La scheda mette a disposizione le principali periferiche del microcontrollore attraverso un'interfaccia adatta alla prototipazione e allo sviluppo di applicazioni embedded.

L'ESP32-S3 integra un processore dual-core e dispone di memoria e periferiche sufficienti per supportare applicazioni caratterizzate dalla presenza simultanea di più attività. Tra le funzionalità disponibili rientrano inoltre la connettività **Wi-Fi** e **BLE**, caratteristiche particolarmente rilevanti per il progetto in esame.

La scheda è stata utilizzata come piattaforma hardware principale per lo sviluppo del firmware, permettendo di gestire contemporaneamente l'acquisizione dei dati provenienti dai sensori, la memorizzazione e le comunicazioni con dispositivi esterni.

4.2 ESP-WROOM-32

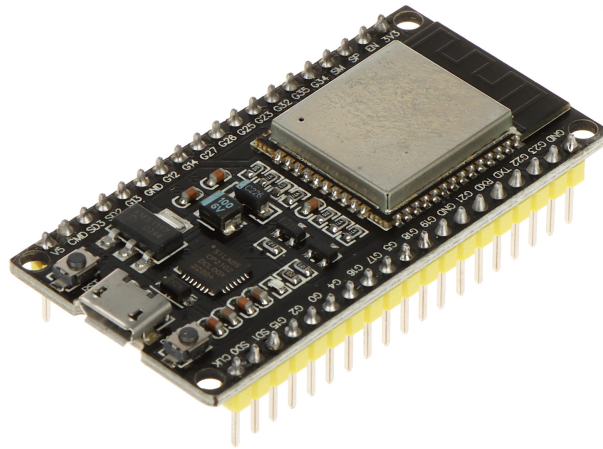


Figura 4.2: Modulo ESP-WROOM-32

L'ESP-WROOM-32 è un modulo basato sulla famiglia di microcontrollori ESP32. A differenza della ESP32-S3-DevKitC-1, che è una scheda di sviluppo, l'ESP-WROOM-32 costituisce principalmente un modulo hardware **compatto** integrabile all'interno di una scheda elettronica.

Il modulo integra il microcontrollore e i componenti necessari al suo funzionamento, oltre alle funzionalità di comunicazione wireless Wi-Fi e BLE. Le sue dimensioni contenute lo rendono particolarmente adatto all'integrazione in dispositivi embedded destinati alla produzione.

Nel progetto, l'ESP-WROOM-32 è stato utilizzato come dispositivo dedicato all'attività di *sniffing* del bus CAN, mantenendolo fisicamente separato dall'unità principale responsabile dell'acquisizione dei dati dai sensori. Questa scelta è stata adottata al fine di ridurre il rischio di danneggiamento dei componenti del sistema principale, considerando l'ambiente elettrico particolarmente critico di una motocicletta, caratterizzato da possibili disturbi e variazioni di tensione dovuti ai sistemi di alimentazione e agli attuatori presenti a bordo. In questo modo, eventuali anomalie o sovratensioni provenienti dall'interfacciamento con il bus CAN rimangono confinate, per quanto possibile, al dispositivo dedicato, preservando gli altri componenti del sistema.

4.3 C++20

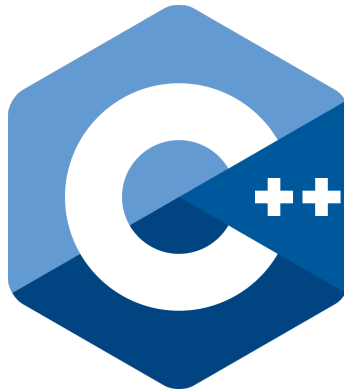


Figura 4.3: Logo del linguaggio C++

Il firmware del dispositivo è stato sviluppato utilizzando il linguaggio C++, facendo riferimento allo standard C++20. La scelta di questo linguaggio è legata principalmente alla necessità di mantenere un controllo diretto sulle risorse hardware e computazionali del microcontrollore.

In un sistema embedded, infatti, memoria, tempo di esecuzione e capacità computazionale sono risorse limitate che devono essere gestite con attenzione. C++ permette di operare a un livello sufficientemente vicino all'hardware, offrendo allo stesso tempo strumenti di astrazione come classi, ereditarietà, incapsulamento e polimorfismo.

4.4 ESP32 Arduino Core

Lo sviluppo del firmware è stato effettuato utilizzando *ESP32 Arduino Core*, un framework che permette di programmare i microcontrollori ESP32 attraverso le API e le modalità di sviluppo tipiche dell'ecosistema Arduino.

Il framework fornisce un livello di astrazione rispetto alle API native di ESP-IDF, rendendo più semplice l'utilizzo delle periferiche e delle funzionalità messe a disposizione dal microcontrollore. Questa caratteristica consente di ridurre la complessità del codice applicativo, mantenendo comunque la possibilità di accedere alle funzionalità specifiche della piattaforma quando necessario.

La scelta di questo framework ha permesso di concentrare lo sviluppo sulle funzionalità richieste dal progetto, evitando di dover gestire direttamente l'intera complessità dell'ESP-IDF.

4.5 FreeRTOS



Figura 4.4: FreeRTOS

FreeRTOS è un kernel real-time progettato per sistemi embedded nei quali diverse attività devono essere eseguite in modo concorrente e rispettando specifici requisiti temporali. Il kernel fornisce principalmente un sistema di gestione delle attività, denominato task scheduling, e una serie di primitive per coordinare l'accesso alle risorse condivise.

Nel firmware sviluppato durante il progetto sono presenti numerose attività che devono operare con frequenze e tempi di risposta differenti. L'acquisizione dei dati provenienti dai sensori, la scrittura sulla memoria microSD e la gestione delle comunicazioni costituiscono infatti attività indipendenti, che devono poter essere eseguite senza bloccare reciprocamente il sistema. L'utilizzo di FreeRTOS permette di suddividere il comportamento del firmware in task indipendenti e lasciare al kernel la gestione dell'accesso al processore.

4.5.1 Sistemi real-time e concorrenza

Per comprendere l'utilità di un kernel come FreeRTOS è utile richiamare brevemente il concetto di concorrenza. In un programma eseguito in maniera sequenziale, le istruzioni vengono normalmente eseguite una dopo l'altra secondo l'ordine stabilito dal programma. Questo modello diventa poco adatto quando il dispositivo deve gestire contemporaneamente più sorgenti di dati o eventi.

In un sistema multitasking, il software viene suddiviso in più attività indipendenti. Quando il microcontrollore dispone di un singolo core, non è possibile eseguire fisicamente più istruzioni appartenenti a task differenti nello stesso istante. Il sistema operativo può tuttavia alternare rapidamente l'esecuzione delle task, dando l'impressione di una loro esecuzione contemporanea.

Il passaggio da una task a un'altra viene indicato come *context switch*. Durante questo processo il sistema conserva le informazioni necessarie per riprendere successivamente l'esecuzione della task sospesa e carica il contesto della task successiva. Nel caso di un processore dotato di più core, alcune task possono invece essere eseguite realmente in parallelo, purché siano disponibili core liberi e la configurazione dello scheduler lo permetta.

4.5.2 Scheduler e priorità

La componente di FreeRTOS responsabile della scelta della task da eseguire è lo scheduler. A ogni task può essere assegnato un livello di priorità, che permette di stabilire quali attività debbano avere precedenza quando più task sono contemporaneamente pronte per l'esecuzione.

Una task che non deve essere eseguita in un determinato momento può inoltre rinunciare temporaneamente al processore, permettendo ad altre attività di utilizzarlo. Questo comportamento è particolarmente importante in sistemi embedded, poiché evita di consumare inutilmente risorse computazionali durante i periodi di attesa.

Il concetto di real-time non implica quindi semplicemente che il sistema sia veloce, ma che le operazioni importanti possano essere completate entro limiti temporali definiti. Nel progetto questo aspetto è rilevante soprattutto per l'acquisizione periodica dei dati dei sensori, dove il rispetto della frequenza di campionamento influenza direttamente la qualità dei dati registrati.

4.5.3 Task

Una task rappresenta un'unità indipendente di esecuzione gestita da FreeRTOS. Ogni task possiede un proprio contesto e viene definita specificando un insieme di parametri, riassunti in Tabella 4.1.

Parametro	Descrizione
Funzione	Punto di ingresso eseguito dalla task, contenente il codice associato all'attività.
Priorità	Valore numerico che determina la precedenza della task rispetto alle altre nella fase di scheduling.
Dimensione dello stack	Quantità di memoria riservata alla task per la gestione di variabili locali e chiamate a funzione.
Parametri	Dati opzionali passati alla funzione al momento della creazione della task.
Core di esecuzione	Nei microcontrollori multi-core, indica su quale core la task può essere schedulata.

Tabella 4.1: Principali parametri di configurazione di una task in FreeRTOS

Nel firmware le task sono state utilizzate per separare funzionalità che presentano caratteristiche temporali differenti. Ad esempio, l'acquisizione dei dati dei sensori può essere eseguita indipendentemente dalle operazioni di scrittura sulla scheda microSD.

Questa suddivisione permette di evitare che un'attività caratterizzata da tempi di esecuzione variabili blocchi un'altra attività che richiede invece un'esecuzione periodica.

4.5.4 Stati delle task

Durante la propria vita una task può assumere differenti stati, in funzione della possibilità di essere eseguita in un determinato momento.

Stato	Descrizione
Running	La task è attualmente in esecuzione su uno dei core disponibili.
Ready	La task è pronta per essere eseguita, ma il processore è momentaneamente assegnato a un'altra task.
Blocked	La task è in attesa del verificarsi di un evento, di una notifica o della scadenza di un intervallo temporale.
Suspended	La task è stata esplicitamente esclusa dallo scheduling e rimane in questo stato fino a una successiva riattivazione.

Tabella 4.2: Principali stati di una task in FreeRTOS

La distinzione tra *Blocked* e *Suspended* è particolarmente importante. Nel primo caso la task rimane in attesa di una condizione specifica, mentre nel secondo viene sospesa esplicitamente e non esiste un evento che possa determinarne autonomamente il risveglio. Questa caratteristica può essere sfruttata, ad esempio, per disabilitare temporaneamente attività che non sono necessarie in una determinata fase del funzionamento del dispositivo.

4.5.5 Race condition e sincronizzazione

La suddivisione del firmware in più task introduce un ulteriore problema: più attività possono avere bisogno di utilizzare contemporaneamente una stessa risorsa.

Si consideri una variabile condivisa x , inizialmente impostata a zero, e due task, A e B , che devono entrambe incrementarla di una unità. Sebbene l'istruzione concettuale sia semplicemente "incrementare x ", a livello macchina l'operazione viene scomposta in tre passaggi elementari: lettura del valore corrente, modifica del valore letto e scrittura del nuovo valore in memoria. Se una task viene interrotta tra uno di questi passaggi e lo scheduler permette all'altra task di operare sulla stessa variabile, si può verificare l'interleaving mostrato in Tabella 4.3.

Istante	Task A	Task B	Valore di x in memoria
t_0	legge x (0)	—	0
t_1	—	legge x (0)	0
t_2	calcola $x + 1$	—	0
t_3	—	calcola $x + 1$	0
t_4	scrive $x = 1$	—	1
t_5	—	scrive $x = 1$	1

Tabella 4.3: Esempio di interleaving che genera una race condition

Nonostante entrambe le task abbiano eseguito un incremento, il valore finale di x è 1 anziché 2, poiché la scrittura effettuata da B sovrascrive quella di A sulla base di un valore letto prima che l'incremento di A venisse reso visibile. Il risultato è una *race condition*, ovvero un comportamento dipendente dall'ordine con cui le task accedono alla risorsa condivisa.

4.5.6 Mutex

Per proteggere le sezioni di codice che accedono a risorse condivise è possibile utilizzare un mutex, abbreviazione di *mutual exclusion*. Il mutex permette a una sola task per volta di entrare nella sezione critica associata alla risorsa protetta.

Il funzionamento è riassunto in Tabella 4.4.

Fase	Descrizione
Richiesta	La task richiede l'acquisizione del mutex prima di accedere alla sezione critica.
Mutex libero	Se il mutex non è occupato, la task lo acquisisce ed entra nella sezione critica.
Mutex occupato	Se il mutex è già occupato da un'altra task, la task richiedente viene posta in stato di attesa.
Rilascio	Al termine dell'accesso alla risorsa, la task rilascia il mutex.
Sblocco	Una delle eventuali task in attesa viene sbloccata e può quindi procedere.

Tabella 4.4: Fasi di utilizzo di un mutex

In questo modo le operazioni che devono essere eseguite senza interferenze da parte delle altre task vengono protette, impedendo accessi concorrenti incompatibili.

Nel firmware questa tecnica risulta utile quando più attività devono accedere a risorse condivise, come strutture dati comuni o periferiche che non possono essere utilizzate contemporaneamente.

4.5.7 Comunicazione tra task

FreeRTOS mette inoltre a disposizione meccanismi specifici per consentire alle task di scambiarsi informazioni senza ricorrere necessariamente a variabili globali condivise.

Tra questi strumenti rientrano principalmente le *queue* e le *task notification*, riassunte e confrontate in Tabella 4.5.

	Queue	Task notification
Contenuto	Trasferisce dati veri e propri tra task	Trasporta un semplice valore o segnale di evento
Modalità	Produttore-consumatore, con più elementi in coda	Diretta, verso una singola task specifica
Overhead	Maggiore, dovuto alla gestione della coda	Ridotto, essendo un meccanismo più leggero
Utilizzo tipico	Scambio di dati acquisiti tra task diverse	Notifica rapida di un evento (es. dato pronto)

Tabella 4.5: Confronto tra queue e task notification in FreeRTOS

Questi meccanismi consentono di progettare componenti maggiormente indipendenti, riducendo la necessità di condividere direttamente lo stato interno delle singole task.

4.6 Expo & React Native



Figura 4.5: Logo di Expo & React Native

React Native è un framework open source che consente di sviluppare applicazioni mobili multiplatforma utilizzando Typescript (o Javascript) e il paradigma dichiarativo tipico di React. React Native traduce i componenti dell'interfaccia in elementi grafici nativi, permettendo di ottenere prestazioni e un aspetto più vicini a quelli di un'applicazione sviluppata nativamente per Android o iOS, pur mantenendo un'unica base di codice condivisa tra le due piattaforme.

Nel progetto React Native è stato utilizzato insieme a *Expo*, un insieme di strumenti che semplifica la configurazione, la compilazione e l'esecuzione di applicazioni React Native. In particolare, lo sviluppo dell'applicazione è stato condotto tramite *development build*, generando ed eseguendo un progetto nativo sul dispositivo di test attraverso i comandi `npx expo run:ios` e `npx expo run:android`, a seconda della piattaforma di destinazione.

4.7 TypeScript

TypeScript è un linguaggio di programmazione sviluppato da Microsoft come soprainsieme (*superset*) di JavaScript, al quale aggiunge un sistema di tipizzazione statica opzionale. Il codice TypeScript viene tradotto (*transpiled*) in JavaScript standard prima dell'esecuzione, mantenendo piena compatibilità con l'ecosistema e le librerie già disponibili per quest'ultimo.

La tipizzazione statica consente di individuare in fase di sviluppo, anziché a runtime, numerosi errori legati a un uso scorretto dei dati, come il passaggio di un parametro di tipo errato a una funzione o l'accesso a una proprietà inesistente di un oggetto. Questo aspetto risulta particolarmente utile in un'applicazione di dimensioni non trascurabili come quella sviluppata nel progetto, dove il numero di componenti, funzioni e strutture dati scambiate tra le diverse parti del codice rende più difficile individuare manualmente questo tipo di errori.

Nel progetto TypeScript è stato utilizzato per l'intero sviluppo dell'applicazione React Native, contribuendo a rendere il codice più leggibile e manutenibile, oltre a facilitare l'integrazione con l'editor di sviluppo grazie al supporto per l'autocompletamento e il controllo dei tipi in tempo reale.

4.8 Git



Figura 4.6: Git

Git è un sistema distribuito per il controllo della versione, utilizzato per registrare e organizzare l'evoluzione del codice sorgente nel tempo. Ogni modifica significativa può essere salvata all'interno della cronologia del progetto, permettendo di recuperare versioni precedenti e analizzare le variazioni introdotte.

Il progetto utilizza Git per la gestione del codice sorgente del firmware, facilitando inoltre lo sviluppo parallelo e la collaborazione tra più sviluppatori.

4.9 PlatformIO



Figura 4.7: PlatformIO

PlatformIO è un ambiente di sviluppo dedicato principalmente alla programmazione di sistemi embedded. Nel progetto è stato utilizzato all'interno di Visual Studio Code per gestire il ciclo di sviluppo del firmware.

Lo strumento automatizza diverse operazioni necessarie durante lo sviluppo, tra cui la compilazione del codice, il caricamento del firmware sulla scheda (*flash*), la gestione delle librerie e delle relative dipendenze e l'utilizzo del monitor seriale.

Un ulteriore vantaggio è rappresentato dalla gestione centralizzata della configurazione del progetto, attraverso la quale è possibile specificare la scheda utilizzata, il framework, le dipendenze e le impostazioni relative alla compilazione.

4.10 L^AT_EX

L^AT_EX è un sistema di composizione tipografica particolarmente diffuso nella produzione di documenti scientifici e accademici. A differenza dei comuni editor visuali, la struttura del documento viene definita attraverso un linguaggio di markup, separando il contenuto dalla sua formattazione.

4.11 PlantUML

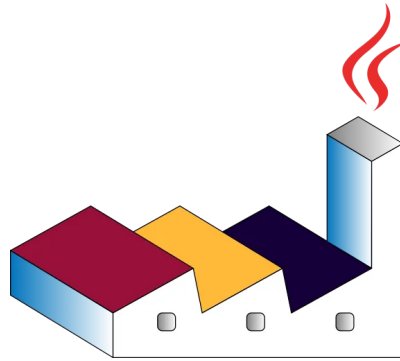


Figura 4.8: Esempio di diagramma generato con PlantUML

PlantUML è uno strumento che consente di generare diagrammi UML a partire da una descrizione testuale. Questa modalità permette di definire la struttura del diagramma direttamente all'interno di file di testo, rendendo semplice la modifica e il mantenimento dei diagrammi nel tempo.

Nel progetto PlantUML è stato utilizzato per la realizzazione dei diagrammi UML impiegati nella documentazione del sistema. La generazione automatica dei diagrammi consente inoltre di mantenere separati il contenuto descrittivo del diagramma e la sua rappresentazione grafica, facilitandone l'aggiornamento durante l'evoluzione del progetto.

5. Progettazione e codifica

Breve introduzione al capitolo

5.1 Tecnologie e strumenti

Di seguito viene data una panoramica delle tecnologie e strumenti utilizzati.

Tecnologia 1

Descrizione Tecnologia 1.

Tecnologia 2

Descrizione Tecnologia 2

5.2 Ciclo di vita del software

5.3 Progettazione

5.3.1 Namespace 1

Descrizione namespace 1.

5.4 Design Pattern utilizzati

5.5 Codifica

6. Verifica e validazione

7. Conclusioni

7.1 Consuntivo finale

7.2 Raggiungimento degli obiettivi

7.3 Conoscenze acquisite

7.4 Valutazione personale

Glossario

Metodologia Agile La metodologia Agile è un approccio alla gestione e allo sviluppo di progetti software basato su iterazioni brevi, collaborazione continua e capacità di adattarsi rapidamente ai cambiamenti dei requisiti. Le attività vengono organizzate in cicli di sviluppo incrementali, durante i quali il team pianifica, realizza e verifica progressivamente le funzionalità del sistema, favorendo il confronto tra i membri del gruppo e il miglioramento continuo del prodotto.. [8](#)

Arduino Arduino è una piattaforma hardware e software open-source basata su microcontrollori, progettata per facilitare lo sviluppo e la prototipazione di sistemi elettronici e applicazioni embedded.. [2](#), [3](#)

Automotive Il termine automotive si riferisce al settore industriale e tecnologico dedicato alla progettazione, allo sviluppo, alla produzione e alla gestione dei veicoli a motore e delle tecnologie ad essi associate.. [2](#)

Bluetooth Low Energy Bluetooth Low Energy è una tecnologia di comunicazione wireless a basso consumo energetico basata sullo standard Bluetooth. È progettata per applicazioni che richiedono lo scambio periodico di piccole quantità di dati mantenendo un ridotto consumo energetico, risultando particolarmente adatta per dispositivi embedded, sensori e sistemi IoT.. [4](#)

bus CAN Il bus Controller Area Network (CAN) è una rete di comunicazione seriale progettata per consentire lo scambio affidabile di dati tra dispositivi elettronici all'interno di un sistema distribuito. È ampiamente utilizzato in ambito automotive per la comunicazione tra le centraline elettroniche del veicolo, grazie alla sua robustezza, alla gestione degli errori integrata e alla capacità di operare in ambienti caratterizzati da elevati livelli di disturbo elettromagnetico.. [4](#)

CAN sniffing Il CAN sniffing è una tecnica di analisi del traffico presente su una rete Controller Area Network (CAN) che consiste nell'acquisizione e registrazione dei messaggi scambiati tra i dispositivi collegati al bus. Questa attività permette di analizzare gli identificativi dei messaggi (*CAN ID*), i dati trasmessi e la frequenza di comunicazione, al fine di individuare la struttura e il significato delle informazioni scambiate tra le centraline elettroniche. In ambito automotive viene utilizzata, ad esempio, per attività di diagnostica, sviluppo di sistemi telemetrici e analisi del comportamento delle ECU.. [4](#)

C++ C++ è un linguaggio di programmazione general-purpose derivato dal linguaggio C, che integra il supporto alla programmazione orientata agli oggetti e a diversi paradigmi di sviluppo. Grazie alle sue elevate prestazioni e alla possibilità di interagire

direttamente con le risorse hardware, viene frequentemente utilizzato nello sviluppo di sistemi embedded e applicazioni dove sono richiesti efficienza e controllo delle risorse.. [4](#)

CSV CSV (*Comma-Separated Values*) è un formato di file testuale utilizzato per rappresentare dati tabellari attraverso una sequenza di valori separati da un delimitatore, generalmente una virgola. Questo formato è ampiamente utilizzato per lo scambio e l'archiviazione di dati strutturati grazie alla sua semplicità e alla compatibilità con numerosi strumenti di elaborazione e analisi dei dati.. [10](#)

Digital Health Con Digital Health si intende l'insieme di tecnologie e soluzioni digitali applicate al settore sanitario, che mirano a migliorare la gestione della salute, la prevenzione delle malattie e l'efficienza dei servizi sanitari. Include strumenti come dispositivi indossabili, applicazioni mobili, telemedicina, intelligenza artificiale e analisi dei dati per supportare la diagnosi, il monitoraggio e il trattamento dei pazienti.. [1](#)

Electronic Control Unit Una Electronic Control Unit (ECU) è una centralina elettronica presente nei veicoli moderni incaricata di controllare e gestire specifiche funzionalità del mezzo, come il controllo del motore, la gestione dell'iniezione o l'acquisizione di dati provenienti dai sensori. Le ECU comunicano frequentemente tra loro attraverso reti interne del veicolo, come il bus CAN, per lo scambio delle informazioni necessarie al funzionamento dei sistemi elettronici.. [4](#)

Embedded Un sistema embedded è un sistema hardware-software dedicato all'esecuzione di funzioni specifiche all'interno di un dispositivo elettronico, spesso caratterizzato da vincoli di prestazioni, memoria e consumo energetico.. [1](#)

Fintech Con Fintech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore finanziario, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

Firmware Software a basso livello che viene eseguito direttamente su un dispositivo hardware, permettendone il controllo e la gestione delle sue componenti elettroniche.. [1](#)

Industrial Internet of Things Con Industrial Internet of Things (IIoT) si intende l'insieme di tecnologie e dispositivi connessi a Internet che vengono utilizzati in ambito industriale per migliorare l'efficienza, la produttività e la sicurezza dei processi produttivi. L'IIoT consente la raccolta e l'analisi dei dati in tempo reale, permettendo alle aziende di prendere decisioni più informate e ottimizzare le operazioni.. [vii](#), [1](#), [3](#)

Internet of Things Insieme di dispositivi fisici connessi alla rete in grado di raccogliere, elaborare e scambiare dati, permettendo il monitoraggio e il controllo remoto delle informazioni.. [2](#)

IMU IMU (*Inertial Measurement Unit*) è un dispositivo elettronico composto generalmente da accelerometri e giroscopi, in grado di misurare rispettivamente l'accelerazione lineare e la velocità angolare di un oggetto. Le IMU vengono utilizzate per rilevare il movimento, l'orientamento e le variazioni di posizione di un dispositivo, risultando fondamentali in numerose applicazioni come sistemi embedded, robotica, veicoli e sistemi di telemetria.. [10](#)

Industria 5.0 Con Industria 5.0 si intende la quinta generazione di industrializzazione, caratterizzata dall'integrazione di tecnologie digitali, l'Intelligenza Artificiale e l'Automazione, per migliorare la produttività e l'efficienza dei processi produttivi.. [1](#)

Insurtech Con Insurtech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore assicurativo, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

LED Il LED (*Light Emitting Diode*) è un componente elettronico a semiconduttore in grado di emettere luce quando è attraversato da una corrente elettrica. Grazie al basso consumo energetico, alla lunga durata e all'elevata efficienza luminosa, è ampiamente utilizzato come indicatore di stato e per applicazioni di illuminazione in dispositivi elettronici ed embedded.. [v](#), [20](#)

Ping-pong buffering Il ping-pong buffering è una tecnica di gestione della memoria utilizzata nei sistemi embedded e nelle applicazioni che richiedono un'acquisizione continua di dati. Consiste nell'utilizzo di due buffer distinti: mentre un buffer viene utilizzato per la scrittura dei nuovi dati acquisiti, l'altro viene elaborato o trasferito dal sistema. Al termine di un ciclo di acquisizione, i due buffer vengono scambiati, permettendo di ridurre i tempi di attesa e garantire un flusso continuo di dati senza interrompere le operazioni di lettura o elaborazione.. [10](#)

RAM La RAM (*Random Access Memory*) è una memoria volatile utilizzata dal sistema per memorizzare temporaneamente dati e istruzioni necessari all'esecuzione dei programmi. Il suo contenuto viene perso allo spegnimento del dispositivo, ma consente un accesso rapido alle informazioni durante il funzionamento del sistema.. [9](#)

Scrum Scrum è un framework appartenente alla metodologia Agile utilizzato per organizzare il lavoro attraverso iterazioni chiamate sprint. Prevede ruoli, eventi e strumenti specifici per pianificare le attività, monitorare l'avanzamento del progetto e favorire la collaborazione tra i membri del team.. [8](#)

Stand-up meeting Lo stand-up meeting è una breve riunione periodica utilizzata nelle metodologie Agile per favorire l'allineamento tra i membri di un team di sviluppo. Durante l'incontro ogni partecipante condivide lo stato delle attività svolte, gli obiettivi da completare e gli eventuali problemi o impedimenti riscontrati. L'obiettivo principale è garantire una comunicazione efficace, individuare tempestivamente eventuali criticità e coordinare il lavoro del gruppo.. [8](#)

UML UML (*Unified Modeling Language*) è un linguaggio di modellazione standard utilizzato nell'ingegneria del software per rappresentare graficamente la struttura, il comportamento e le interazioni di un sistema. Attraverso diversi tipi di diagrammi, UML supporta le attività di analisi, progettazione e documentazione del software, facilitando la comunicazione tra gli sviluppatori e gli stakeholder coinvolti.. [10](#)

Bibliografia

Testi

James P. Womack, Daniel T. Jones. *Lean Thinking, Second Editon*. Simon & Schuster, Inc., 2010 (cit. a p. [vii](#)).

Articoli

Einstein, Albert, Boris Podolsky e Nathan Rosen. «Can Quantum-Mechanical Description of Physical Reality be Considered Complete?» In: *Physical Review* 47.10 (1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](#) (cit. a p. [vii](#)).

Sitografia

Manifesto Agile. URL: <http://agilemanifesto.org/iso/it/> (cit. alle pp. 3, vii).

Questa pagina verrà momentaneamente utilizzata per vedere gli esempi di utilizzo di figure, tabelle, glossario e citazioni ecc...

Glossario

Esempio di utilizzo di un termine nel glossario *Industrial Internet of Things_G*. La prima volta che usi un acronimo usa "acronimo" per vedere l'acronimo esteso, da lì in poi usa la versione "acronimo".

Citazioni

Esempio di citazione direttamente nel testo *Manifesto Agile*.

Citazioni a piè pagina

Esempio di citazione nel piè di pagina¹.

Introduzione all'idea dello stage².

¹Daniel T. Jones James P. Womack. *Lean Thinking, Second Edition*. Simon & Schuster, Inc., 2010.

²Albert Einstein, Boris Podolsky e Nathan Rosen. «Can Quantum-Mechanical Description of Physical Reality be Considered Complete?» In: *Physical Review* 47.10 (1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](https://doi.org/10.1103/PhysRev.47.777).